



Remote Control Specification

Remote Control Specification for Ethernet

Version	0.4.3/1
Date	2026-04-19
Status	Draft
Restriction Level	OPEN Members Only

VERSION AND RESTRICTION HISTORY OF THIS DOCUMENT

VERSION/REVISION	DESCRIPTION	RESTRICTION LEVEL	DATE
0.1	Initial Version	Internal	
0.2	Chapter 8 approved	Internal	10.02.2025
0.3	Voted TC18 draft basis	Internal	10.12.2025
0.4.2	Reformatted and review comments included Temporary backlog for GBB change tracking Added execution priorities in chapter 11.10	Internal	19.02.2026
0.4.2/3	Chapter 11.9 marked explicitly as an optional feature	Internal	09.04.2026
0.4.3/1	Proposed changes are marked with comments	internal	19.04.2026

CHAIR AND VICE CHAIR

CHAIR	NAME	ORGANIZATION
OPEN Alliance TC18 Chair	Gumersindo Veloso Cauce	BMW
OPEN Alliance TC18 Vice Chair	Adriaan Niess	Bosch

EDITOR

NAME	ORGANIZATION
Gumersindo Veloso Cauce	BMW
Bernd Elend	NXP

CONTRIBUTORS

NAME	ORGANIZATION

Table of Contents

VERSION AND RESTRICTION HISTORY OF THIS DOCUMENT	2
CHAIR AND VICE CHAIR.....	2
EDITOR	2
CONTRIBUTORS	2
1 OPEN SPECIFICATION OWNERSHIP AND USAGE RIGHTS	6
2 RIGHTS AND USAGE RESTRICTIONS SPECIFIC TO OPEN ALLIANCE MEMBERS	6
2.1 Rights and Usage Restrictions Specific to Non-members of OPEN Alliance	6
3 TERMS APPLICABLE TO BOTH MEMBERS AND NON-MEMBERS OF OPEN ALLIANCE ..	7
3.1 Patents, Trademarks, and other Rights:.....	7
3.2 Disclaimers and Limitations of Liability:	7
3.3 Compliance with Laws and Regulations:	7
3.4 Automotive Applications Only:.....	8
3.5 Right to Withdraw or Modify:	8
4 ABBREVIATION/SYMBOLS	9
5 TERMS AND DEFINITIONS.....	11
6 NORMATIVE REFERENCES.....	12
7 FUNDAMENTALS	ERROR! BOOKMARK NOT DEFINED.
 7.1 Purpose of this Document	Error! Bookmark not defined.
 7.2 Structure of this Document.....	Error! Bookmark not defined.
8 INTRODUCTION TO REMOTE CONTROL	12
8.1 Motivation.....	13
8.2 General Objectives.....	14
8.3 Scope	15
9 REMOTE CONTROL SYSTEM.....	16
9.1 RC Node	16
9.2 RC Client	16
9.3 RC Server.....	16
9.4 RC Endpoint.....	17
9.5 RC Edge Node	18
9.6 Communication layers	18
9.7 Network.....	18

9.8 RC CONTROLLER	18
10 REMOTE CONTROL PROTOCOL	19
10.1 Overview	19
10.2 Networking.....	19
11 USAGE OF IEEE1722 FOR RCP	20
11.1 Overview	20
11.2 Time synchronization.....	23
11.2.1 General Information.....	23
11.2.2 Capture time stamps (message_timestamp)	23
11.2.3 Presentation time stamp (avtp_timestamp)	23
11.3 The usage of the op bit in requests	24
11.4 The usage of evt bits in requests	25
11.4.1 Request specific usage of evt-bit and op-bit	26
11.5 The usage of the err bit to cancel a request	29
11.6 The usage of the byte_msg_payload in requests	29
11.7 Compound operations – Sequences based request execution	30
11.8 Triggered execution of requests	32
11.9 Chain operation (Optional feature).....	33
11.10 Priorities in execution	36
12 RC SERVER	38
12.1 Overview	38
12.2 Lifecycle	38
12.2.1 Lifecycle of the RC Phy.....	38
12.2.2 Lifecycle of the Remote-Control Framework.....	39
12.3 Power-On / Wake-Up / Start-Up behavior	40
12.4 Sleep behavior	40
12.5 Discovery	40
12.6 Configuration	41
12.6.1 Configuration in RC Phy's hw_unconfigured state.....	41
12.6.2 Configuration in RC Phys's hw_configured state	41
12.6.3 Register map - General part	42
12.6.4 HW pin mapping configuration	44
12.6.5 Client for request stream configuration	46
12.6.6 Endpoint ID and communication configuration.....	48
12.6.7 Endpoint group configuration.....	49
12.6.8 Response stream configuration	51
12.6.9 Sequencer state registers	52
12.6.10 Network interface configuration	53
12.6.11 PTP configuration.....	54
12.7 Communication handling	55
12.7.1 Network interfaces	55

12.7.2 Reception of frames.....	55
12.7.3 Handling of requests, responses, acknowledges and notifications	55
13 ENDPOINTS	58
13.1 Overview.....	58
13.2 Generic part of endpoint register map	58
13.3 Request validation.....	60
13.4 Endpoint status information	60
13.5 Endpoint specific usage of evt-bits and byte_msg_payloads	62
13.6 End-to-End data protection (safe points)	64
13.6.1 End-to-End protection (EP groups)	64
13.7 Types of endpoints	65
13.7.1 Endpoint type: RC Server	65
13.7.2 Endpoint type: Wakeup control	68
13.7.3 Endpoint type: SPI (SPI controller)	70
13.7.4 Endpoint type: GPIO	73
13.7.5 Endpoint type: PWM_OUT	75
13.7.6 Endpoint type: PWM_IN	77
13.7.7 Endpoint type: I ² C (I ² C Controller)	79
13.7.8 Endpoint type: UART	80
13.7.9 Endpoint type: ADC.....	82
13.7.10 Endpoint type: LIN commander.....	85
13.7.11 Endpoint type: CAN controller	86
13.7.12 Endpoint type: ISELED.....	88

OPEN Alliance Specification Copyright Notice and Disclaimer

1 OPEN SPECIFICATION OWNERSHIP AND USAGE RIGHTS

As between OPEN Alliance and OPEN Alliance Members whose contributions were incorporated in this OPEN Specification (the "Contributing Members"), the Contributing Members own the worldwide copyrights in and to their given contributions. Other than the Contributing Members' contributions, OPEN Alliance owns the worldwide copyrights in and to compilation of those contributions forming this OPEN Specification. For OPEN Alliance Members (as that term is defined in the OPEN Alliance Bylaws), OPEN Alliance permits the use of this OPEN Specification on the terms in the OPEN Alliance Intellectual Property Rights Policy and the additional applicable terms below. For non-members of OPEN Alliance, OPEN Alliance permits the use of this OPEN Specification on the terms in the OPEN Alliance Specification License Agreement (available here: <http://www.opensig.org/Automotive-Ethernet-Specifications/>) and the additional applicable terms below. The usage permissions referenced and described here relate only to this OPEN Specification and do not include or cover a right to use any specification published elsewhere and referred to in this OPEN Specification. By using this OPEN Specification, you hereby agree to the following terms and usage restrictions:

2 RIGHTS AND USAGE RESTRICTIONS SPECIFIC TO OPEN ALLIANCE MEMBERS

FOR OPEN ALLIANCE MEMBERS ONLY: In addition to the usage terms and restrictions granted to Members in the OPEN Alliance Intellectual Property Rights Policy, Members' use of this OPEN Specification is subject this Copyright Notice and Disclaimer. Members of OPEN Alliance have the right to use this OPEN Specification solely (i) during the term of a Member's membership in OPEN Alliance and subject to the Member's continued membership in good standing in OPEN Alliance; (ii) subject to the Member's continued compliance with the OPEN Alliance governance documents, Intellectual Property Rights Policy, and the applicable OPEN Alliance Promoter or Adopter Agreement, as applicable; and (iii) for internal business purposes and solely to use the OPEN Specification for implementation of this OPEN Specification in the Member's products and services, but only so long as Member does not distribute, publish, display, or transfer this OPEN Specification to any third party, except as expressly set forth in Section 11 of the OPEN Alliance Intellectual Property Rights Policy. Except and only to the extent required to use this OPEN Specification internally for implementation of this OPEN Specification in Member's products and services, Member shall not modify, alter, combine, delete portions of, prepare derivative works of, or create derivative works based upon this OPEN Specification. If Member creates any modifications, alterations, or other derivative works of this OPEN Specification as permitted to use the same internally for implementation of this OPEN Specification in Member's products and services, all such modifications, alterations, or other derivative works shall be deemed part of, and licensed to such Member under the same restrictions as, this OPEN Specification. For the avoidance of doubt, Member shall not include all or any portion of this OPEN Specification in any other technical specification or technical material, product manual, marketing material, or any other material without OPEN Alliance's prior written consent. All rights not expressly granted to Member in the OPEN Alliance Intellectual Property Rights Policy are reserved;

2.1 Rights and Usage Restrictions Specific to Non-members of OPEN Alliance

FOR NON-MEMBERS OF OPEN ALLIANCE ONLY: Use of this OPEN Specification by anyone who is not a Member in good standing of OPEN Alliance is subject to your agreement to the OPEN Alliance Specification License Agreement (available here: <http://www.opensig.org/Automotive-Ethernet-Specifications/>) and the additional terms in this Copyright Notice and Disclaimer. Non-members have the right to use this OPEN Specification solely (i) subject to the non-member's continued compliance with the OPEN Alliance Specification License Agreement; and (ii) for internal business purposes and solely to use the OPEN Specification for implementation of this OPEN Specification in the non-member's products and services, but only so long as non-member does not distribute, publish, display, or transfer this OPEN Specification to any third party, unless and only to the extent expressly set forth in the OPEN

Alliance Specification License Agreement. Except and only to the extent required to use this OPEN Specification internally for implementation of this OPEN Specification in non-member's products and services, non-member shall not modify, alter, combine, delete portions of, prepare derivative works of, or create derivative works based upon this OPEN Specification. If non-member creates any modifications, alterations, or other derivative works of this OPEN Specification as permitted to use the same internally for implementation of this OPEN Specification in non-member's products and services, all such modifications, alterations, or other derivative works shall be deemed part of, and licensed to such non-member under the same restrictions as, this OPEN Specification. For the avoidance of doubt, non-member shall not include all or any portion of this OPEN Specification in any other technical specification or technical material, product manual, marketing material, or any other material without OPEN Alliance's prior written consent. All rights not expressly granted to non-member in the OPEN Alliance Specification License Agreement are reserved.

3 TERMS APPLICABLE TO BOTH MEMBERS AND NON-MEMBERS OF OPEN ALLIANCE

3.1 Patents, Trademarks, and other Rights:

OPEN Alliance has received no Patent Disclosure and Licensing Statements related to this OPEN Specification. Therefore, this OPEN Specification contains no specific disclaimer related to third parties that may require a patent license for their Essential Claims. Having said that, the receipt of this OPEN Specification shall not operate as an assignment of or license under any patent, industrial design, trademark, or other rights as may subsist in or be contained in or reproduced in this OPEN Specification; and the implementation of this OPEN Specification could require such a patent license from a third party. You may not use any OPEN Alliance trademarks or logos without OPEN Alliance's prior written consent.

3.2 Disclaimers and Limitations of Liability:

THIS OPEN SPECIFICATION IS PROVIDED ON AN "AS IS" BASIS, AND ALL REPRESENTATIONS, WARRANTIES, AND GUARANTEES, EITHER EXPLICIT, IMPLIED, STATUTORY, OR OTHERWISE, ARE EXCLUDED AND DISCLAIMED UNLESS (AND THEN ONLY TO THE EXTENT THEY ARE) MANDATORY UNDER LAW. ACCORDINGLY, OPEN ALLIANCE AND THE CONTRIBUTING MEMBERS MAKE NO REPRESENTATIONS OR WARRANTIES OR GUARANTEES WITH REGARD TO THIS OPEN SPECIFICATION OR THE INFORMATION (INCLUDING ANY SOFTWARE) CONTAINED HEREIN. OPEN ALLIANCE AND ALL CONTRIBUTING MEMBERS HEREBY EXPRESSLY DISCLAIM ANY AND ALL SUCH EXPRESS, IMPLIED, STATUTORY, AND ALL OTHER REPRESENTATIONS, WARRANTIES, AND GUARANTEES, INCLUDING WITHOUT LIMITATION ANY AND ALL WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE OR USE, TITLE, NON-INFRINGEMENT OF OR ABSENCE OF THIRD PARTY RIGHTS, AND/OR VALIDITY OF RIGHTS IN THIS OPEN SPECIFICATION; AND OPEN ALLIANCE AND THE CONTRIBUTING MEMBERS MAKE NO REPRESENTATIONS AS TO THE ACCURACY OR COMPLETENESS OF THIS OPEN SPECIFICATION OR ANY INFORMATION CONTAINED HEREIN. WITHOUT LIMITING THE FOREGOING, OPEN ALLIANCE AND/OR CONTRIBUTING MEMBERS HAS(VE) NO OBLIGATION WHATSOEVER TO INDEMNIFY OR DEFEND YOU AGAINST CLAIMS RELATED TO INFRINGEMENT OR MISAPPROPRIATION OF INTELLECTUAL PROPERTY RIGHTS.

OPEN ALLIANCE AND CONTRIBUTING MEMBERS ARE NOT, AND SHALL NOT BE, LIABLE FOR ANY LOSSES, COSTS, EXPENSES, OR DAMAGES OF ANY KIND WHATSOEVER (INCLUDING WITHOUT LIMITATION DIRECT, INDIRECT, SPECIAL, INCIDENTAL, CONSEQUENTIAL, PUNITIVE, AND/OR EXEMPLARY DAMAGES) ARISING IN ANY WAY OUT OF USE OR RELIANCE UPON THIS OPEN SPECIFICATION OR ANY INFORMATION HEREIN. NOTHING IN THIS DOCUMENT OPERATES TO LIMIT OR EXCLUDE ANY LIABILITY FOR FRAUD OR ANY OTHER LIABILITY WHICH IS NOT PERMITTED TO BE EXCLUDED OR LIMITED BY OPERATION OF LAW.

3.3 Compliance with Laws and Regulations:

NOTHING IN THIS DOCUMENT OBLIGATES OPEN ALLIANCE OR CONTRIBUTING MEMBERS TO PROVIDE YOU WITH SUPPORT FOR, OR RELATED TO, THIS OPEN SPECIFICATION OR ANY IMPLEMENTED

PRODUCTS OR SERVICES. NOTHING IN THIS OPEN SPECIFICATION CREATES ANY WARRANTIES OR GUARANTEES, EITHER EXPRESS OR IMPLIED, STATUTORY OR OTHERWISE, REGARDING ANY LAW OR REGULATION. OPEN ALLIANCE AND CONTRIBUTING MEMBERS EXPRESSLY DISCLAIM ALL LIABILITY, INCLUDING WITHOUT LIMITATION, LIABILITY FOR NONCOMPLIANCE WITH LAWS, RELATING TO USE OF THE OPEN SPECIFICATION OR INFORMATION CONTAINED HEREIN. YOU ARE SOLELY RESPONSIBLE FOR THE COMPLIANCE OF IMPLEMENTED PRODUCTS AND SERVICES WITH ANY SUCH LAWS AND REGULATIONS, AND FOR OBTAINING ANY AND ALL REQUIRED AUTHORIZATIONS, PERMITS, AND/OR LICENSES FOR IMPLEMENTED PRODUCTS AND SERVICES RELATED TO SUCH LAWS AND REGULATIONS WITHIN THE APPLICABLE JURISDICTIONS.

IF YOU INTEND TO USE THIS OPEN SPECIFICATION, YOU SHOULD CONSULT ALL APPLICABLE LAWS AND REGULATIONS. COMPLIANCE WITH THE PROVISIONS OF THIS OPEN SPECIFICATION DOES NOT CONSTITUTE COMPLIANCE TO ANY APPLICABLE LEGAL OR REGULATORY REQUIREMENTS. IMPLEMENTERS OF THIS OPEN SPECIFICATION ARE SOLELY RESPONSIBLE FOR OBSERVING AND COMPLYING WITH THE APPLICABLE LEGAL AND REGULATORY REQUIREMENTS. WITHOUT LIMITING THE FOREGOING, YOU SHALL NOT USE, RELEASE, TRANSFER, IMPORT, EXPORT, AND/OR RE-EXPORT THIS OPEN SPECIFICATION OR ANY INFORMATION CONTAINED HEREIN IN ANY MANNER PROHIBITED UNDER ANY APPLICABLE LAWS AND/OR REGULATIONS, INCLUDING WITHOUT LIMITATION U.S. EXPORT CONTROL LAWS.

3.4 Automotive Applications Only:

Without limiting the foregoing disclaimers or limitations of liability in any way, this OPEN Specification was developed for automotive applications only. This OPEN Specification has neither been developed, nor tested for, non-automotive applications.

3.5 Right to Withdraw or Modify:

OPEN Alliance reserves the right to (but is not obligated to) withdraw, modify, or replace this OPEN Specification at any time, without notice.

© 2024 OPEN Alliance. This document also contains contents, the copyrights of which are owned by third parties who are OPEN Alliance Contributing Members. Unauthorized Use Strictly Prohibited. All Rights Reserved.

1 4 ABBREVIATION/SYMBOLS

MAC	Media Access Control
I/O	Input/Output
RC	Remote Control
UTP	Underlying Transport Protocol
PDU	Protocol Data Unit
AVTPDU	Audio Video Transport PDU
MTU	Maximum Transmission Unit
EP	Endpoint
LC	Lifecycle
EP0	Endpoint 0 (which is the RC Server itself)
PCB	Printed-circuit Board
SPI	Serial Peripheral Interface
PWM	Pulse Width Modulation
ADC	Analog to Digital Converter
DAC	Digital to Analog Converter
CAN	Controller Area Network
ISELED	Integrated smart embedded LED (registered trademark from innova Semiconductors)
LED	Light emitting diode
HW	Hardware
SW	Software
RCP	remote control protocol
CMD	Command
LEN	Length
TDC	time delay compensation (used in CAN FD)
R/W	read / write

IO	input-output
ACF	audio-video control format
GBB	generic byte bus
ABB	abbreviated byte bus
CRC	cyclic redundancy check
TSCF	time synchronous control frame
NTSCF	non-time synchronous control frame
EVT	event
Ptr	pointer
FUNC	functional
API	application programming interface
BB	byte bus
WD	watchdog
PTP	precision time protocol
PL	physical layer
UTM	underlying transport mechanism

1
2

5 TERMS AND DEFINITIONS

For the purposes of this document, the following terms and definitions apply.

ISO and IEC maintain terminological databases for use in standardization at the following addresses:

1. **ISO Online browsing platform:** available at <https://www.iso.org/obp>

2. **IEC Electropedia:** available at <http://www.electropedia.org/>

Remote Control System Definitions:

RC System	A Remote Control System consists of a collection of Remote Control Nodes connected via a bridged or routed Network.
RC Node	A Remote Control Node can either contain a Remote Control Client or a Remote Control Server.
Edge Node	A Remote Control Edge node contains a Remote Control Server. Additional functionalities may be supported such as like PTP and MACsec key agreement.
RC Client	A Remote Control Client is part of the Node enabled to control and communicate with the Endpoints through their respective Servers.
RC Server	A Remote Control Server offers Endpoints (SPI, I2C, GPIO, ... Interfaces) which can be accessed/controlled by the client.
Endpoint	Each RCP addressable component in the RC Server.
OEM IP	Communication logic provided by and restricted to an OEM. Not in the scope of this specification and irrelevant for it.
Commodity IP	Standardized communication logic provided by network participants other than the OEM. Not in the scope of this specification and irrelevant for it
Application logic	Software component interacting with the RC Client (e.g. application device driver, ...).

Note: Unless otherwise specified, the Terms 'System', 'Node', ... refer to the Remote Control System, Remote Control Node, ...

Remote Control Protocol Definitions:

RCP Message	Synonymous for RCPPDU.
RCP Frame	Construct made of one RCP Message plus UTP Headers and Footers.
Operation	Use of a specific functionality on the targeted Endpoint

1

2 **6 NORMATIVE REFERENCES**

3 IEEE1722-2025 **Transport Protocol for 2 Time-Sensitive Applications in Bridged Local Area Networks**

4 **7**

5 *Paragraph removed*

6

8 INTRODUCTION TO REMOTE CONTROL

Remote Control (RC) is an Ethernet based Client-Server-Architecture which RC Clients, to remotely control a device connected to a Remote Control **Server's Endpoint**, while allowing to allocate most, if not all, of the related application logic on the Client side alone.

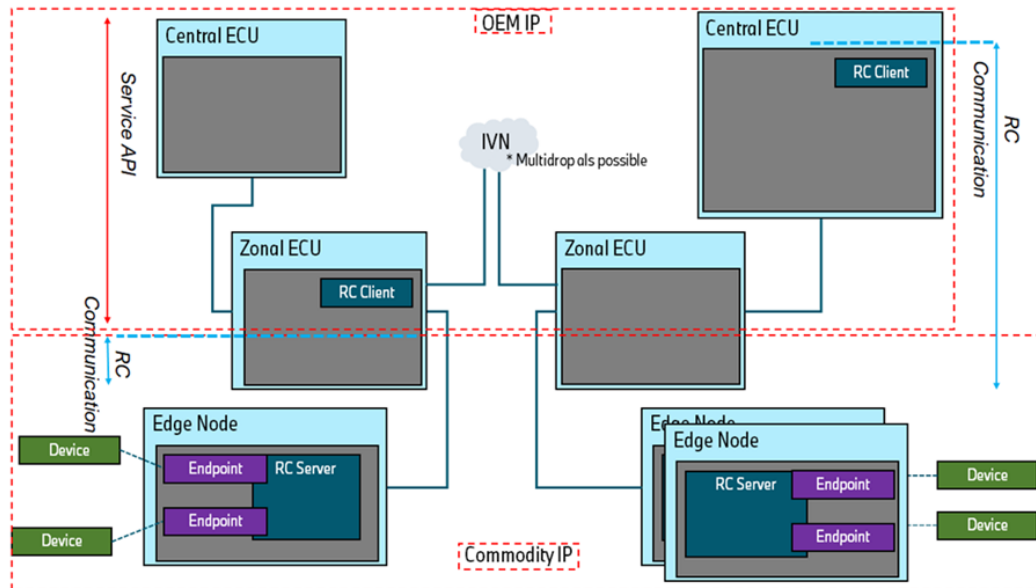


Figure 1 - General depiction of a Remote Control System

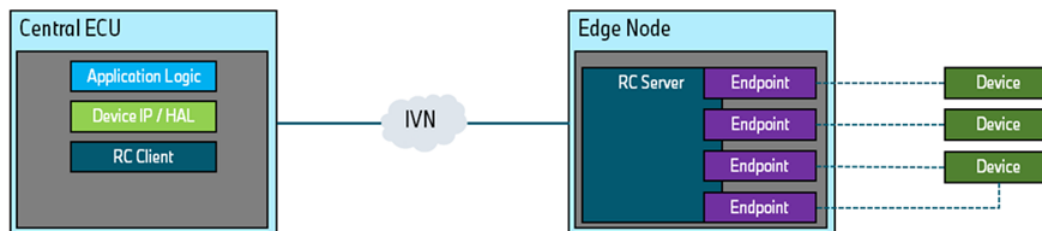


Figure 2 - Basic internal view of Remote Control participants

It is distinguished between the Remote Control **System** and the Remote Control **Protocol**, the latter being the means under which components in the Remote Control System can interact.

8.1 Motivation

The Remote Control (RC) System enables a clear separation between OEM Intellectual Property (IP) and Commodity IP (off-the-shelf components). This is achieved by centralizing application logic in High-Performance Computers (HPCs), thereby reducing the complexity of peripheral components.

The RC Protocol serves as a standardized interface for peripherals, ensuring easy effortless integration into the RC System. This will enable greater flexibility in supplier selection, strengthening supply chain resilience. Additionally, the protocol provides cost-efficient access to devices connected to peripheral ECUs, supporting the transition to a zonal architecture.

8.2 General Objectives

These are the objectives of Remote Control Protocol:

1. Remote Access to Low-level Interfaces

- The RCP shall allow client applications to access low-level peripheral interfaces physically located on other ECUs via a communication network.
- Like with local interfaces, the RCP shall offer full access to data transfer operations, configuration parameters, status information as well as other functionalities.
- Implementation specific details of remotely controlled interfaces should not be exposed in order to provide an abstraction over the actual hardware.

2. Homogeneous Integration in existing peripheral IVN

- The RCP shall facilitate homogenous integration of diverse ECUs into in-vehicle networks through standardized remotely controlled Endpoints providing an abstraction over physical hardware interfaces.

3. Compensate Limitations of Distributed Systems

- The RCP shall provide means to optimize communication based on latency and/or bandwidth constraints. For example, by aggregating or not aggregating multiple RC operations within a single RCPDU.
- It shall be possible to synchronize sensors/actuators and compensate for jitter in the network.
- Additionally, communication via unreliable communication channels shall be possible and support for safety measures shall be provided.

4. Hardware/Software decoupling

- The RCP should enable interchangeability of hardware- and software modules from different suppliers.
- The RCP shall provide an unambiguous definition of behavior. communication, error handling, etc.

5. Backward and Forward compatibility

- The RCP shall be able to differentiate between messages from newer and older versions.
- Implementations of older versions of RCP shall safely interact with newer versions by ignoring or gracefully handling new message types.

8.3 Scope

This specification contains all components and mechanisms involved in the Remote Control communication. External Participants or Devices connected to Endpoints are not part of Remote Control.

How an RCP compliant product is implemented, as well as any considerations towards hardware or software implementing Remote Control, is not in the scope of this document.

The RCP version specified in this document covers RC operations for the following interfaces:

~~SPI, I2C, GPIO, UART.~~

1. Digital I/O
2. Analog-to-Digital Converter (ADC)
3. Pulse-width modulation (PWM)
4. Serial Peripheral Interface (SPI)
5. Inter-Integrated Circuit (I2C)
6. Universal Asynchronous Receiver Transmitter (UART)

9 REMOTE CONTROL SYSTEM

This specification contains all components and mechanisms involved in the Remote Control communication. External Participants or Devices connected to Endpoints are not part of Remote Control.

How a Remote Control compliant product is implemented, as well as any considerations towards hardware or software implementing Remote Control, is not in the scope of this document.

An RC System as depicted in Figure 3 is a collection of RC Nodes connected through an RC Network. Each RC Node is either be an RC Client or an RC Server. A System can be built from any number of Nodes, but at least one client and one server.

An RC Server contains a set of RC Endpoints offering functions to be accessed by an RC Client.

The System enables Clients to control and communicate with Endpoints through their respective Servers using the Remote Control Protocol (RCP).

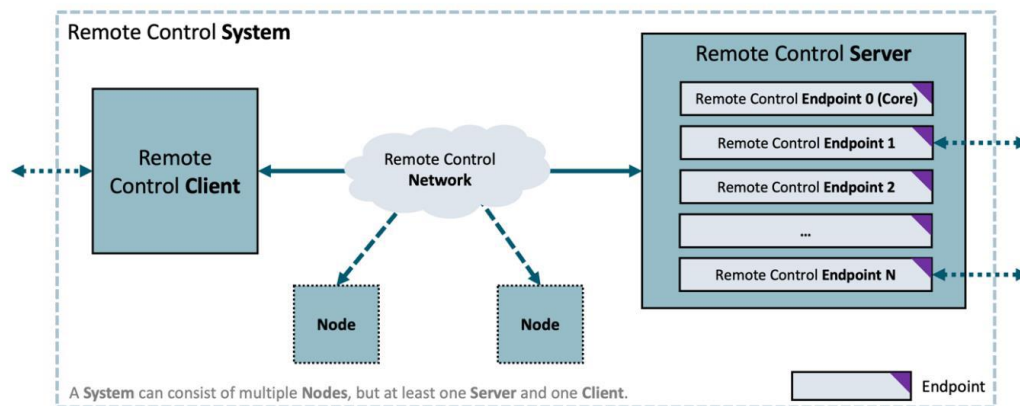


Figure 3 - Remote Control System

9.1 RC Node

The RCP defines two types of Nodes: Clients and Servers. All Nodes (both Clients and Servers) are uniquely identified within the system. The exact details are explained in chapter 10.

9.2 RC Client

RC Clients are the software components communicating with RC Servers and remote controlling Devices by invoking functions on RC Endpoints over the network through RC Requests. Clients might also communicate with external participants to the RC System such as services, device drivers or other software components.

Since Clients are most likely software-based implementations by OEMs, this specification does not specify any of the implementation specific details.

9.3 RC Server

An RC Server handles requests which are meant to be processed by its Endpoints as well as responses and notifications generated by the Endpoints. Moreover, an RC Server provides access to its own configuration and the configuration of all its Endpoints by exposing itself as a logical Endpoint.

Discovery of RC servers is done by reading (part of) the RCP server's configuration.. Multiple Servers can be discovered at once by broadcasting read requests in case none of the RC Server's MAC addresses are known. The RC Server exposes

itself as an Endpoint to the network. The Server is sending responses, acknowledges and notifications created by its endpoints to Clients.

The RC Server may receive data from a connected device through one of its interfaces. This data is processed by the corresponding Endpoint, which ensures that the RC Response handler receives all relevant information in its responder queue, including identification of the originating interface, the input data itself, an optional timestamp, and the association with the initiating request.

A distinction is made between responses and notifications. Notifications are generated by requests that can be executed repeatedly and may therefore produce multiple outputs. The RC Response handler at the protocol layer manages the communication, for example by aggregating multiple responses and/or notifications into a single frame. This layer also handles the information required by the RC Client to detect lost messages.

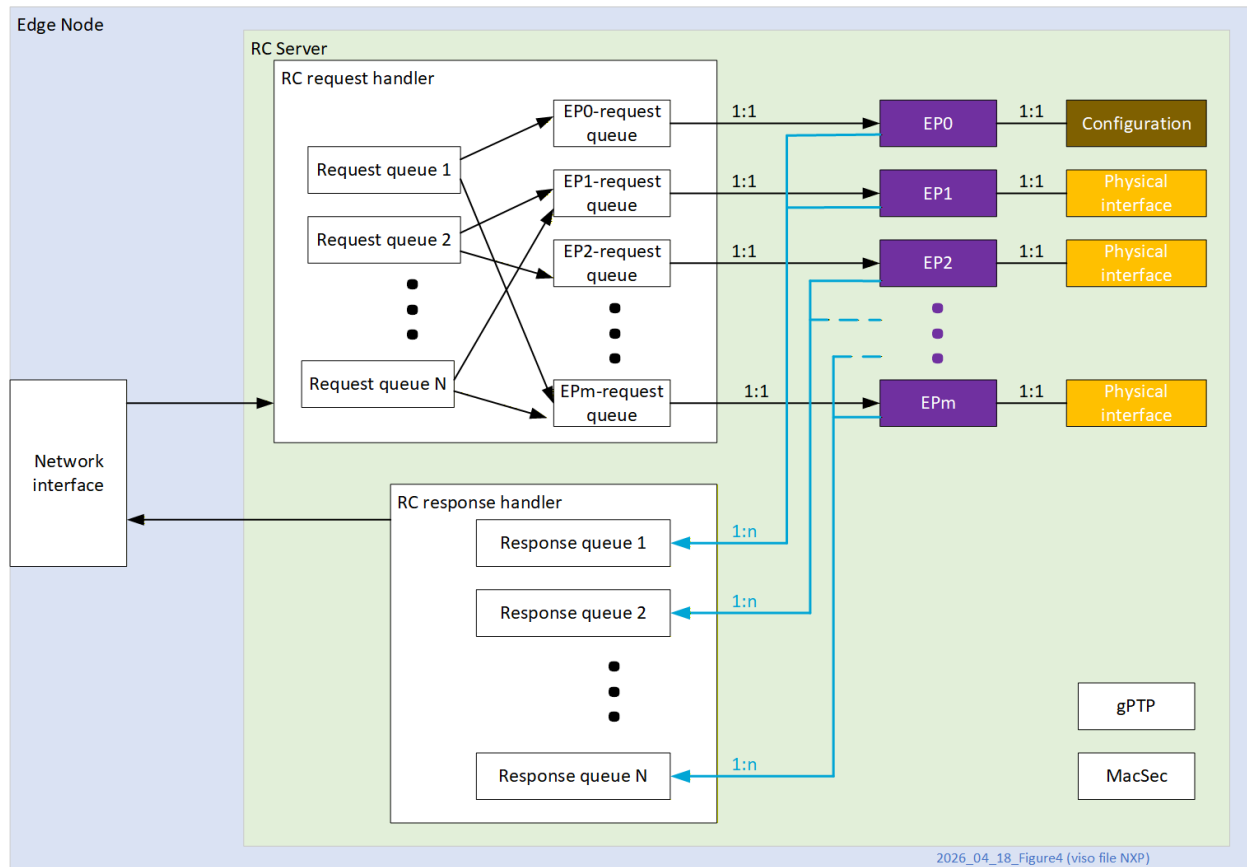


Figure 4 – Exemplary Edge Node with RC Server block diagram

9.4 RC Endpoint

An Endpoint is either a Physical Endpoint providing an abstraction over a physical interface or a pure Logical Endpoint. Each Endpoint has a system-wide unique discoverable identifier (Endpoint description) and offers functions to update runtime configuration parameters, read statistic counters or invoke Endpoint specific functions, for example in the case of Physical Endpoints, updating the logic level of physical output pins or initiating bus transfers.

Physical Endpoints might not exactly model their corresponding hardware IP but provide an abstraction to allow for homogenous integration into Client software similar than a MCAL (MCU abstraction layer). The RC Server itself is treated

as Endpoint 0 (EP0) for configuration and status handling, while optional sleep/wake-up functionality is handled by Endpoint 1 (EP1).

To interact with an Endpoint, clients must know its capabilities and configurable parameters. Each Endpoint maintains a dedicated request queue, executes valid requests received from the RC Server. The RC Request handler validates incoming requests, routes them to the appropriate Endpoint queues, and supports various request types (standard, timed, wait, trigger-based, or compound).

Responses from Endpoints are placed into responder queues and processed by the RC Response handler, which formats and fragments messages as needed according to network frame size limitations before sending them back to Clients.

9.5 RC Edge Node

The RC Edge Node is the core component of the RC System. It is composed of an RC Server and might support additional communication layers and protocols besides the RCP itself. An RC Server is addressable on the network and provides capabilities to configure/control the overall RC Edge Node including its physical interfaces exposed as RC endpoints.

Figure 4 depicts a block diagram of an Edge Node which shows the physical network interface and physical IO as well as the logical structure of the RC Server, RC Request- and Response handlers and Endpoints.

9.6 Communication layers

The RC Protocol shall be network independent.

The RC Protocol frame format shall be based on IEEE1722 and make use of the ACF-type byte bus and byte bus brief using the ACF-message Abbreviated Byte Bus (ABB) and Generic Byte Bus (GBB) for communication purposes (see Usage of IEEE1722 for RCP). Within this document also referenced as ACF_ABB and ACF_GBB.

The RC Server shall handle requests from the RC Client and assign them to the correct endpoints.

The RC Server shall support multiple requests in one frame.

9.7 Network

The network is not part of the remote control system. However, it is responsible for the correct delivery of remote control messages from Clients to Servers and vice versa.

9.8

Paragraph removed

10 REMOTE CONTROL PROTOCOL

10.1 Overview

IEEE Draft Std 1722™ P1722-2016-Rev/D9 is used as base for the remote control protocol (RCP).

IEEE1722 was originally made for audio/video transportation but has been extended to also carry control data. It can be used as a layer-2 protocol, which is independent from the physical layer below. IEEE1722 can also be used in IP-networks.

The data traffic is organized in "streams". Each process data unit (PDU) sent over the network has an identification of the stream it belongs to. The network is responsible to route the stream to the intended recipients.

The stream identification includes a unique stream id identifier and the senders MAC address, which supports scalability in network size without the need to re-organize client ids, since MAC addresses are unique as per definition.

A PDU according to IEEE1722 is a so-called "AVTPDU" (audio/video-transport PDU). In Ethernet frame the presence of an AVTPDU is marked by an EtherType value of 0x22F0. Encapsulation of 1722 frames in IP/UDP and port usage is described in Annex J - IP Encapsulation of the IEEE Draft Std 1722™ P1722- 2016-Rev/D7.

The IEEE1722 protocol begins with a header, which is either the Non-Time-Synchronous Control Format (NTSCF) or the Time-Synchronous Control Format (TSCF) header.

The TSCF header includes a presentation time that defines the time at which requests in the AVTPDU become due for execution, while the NTSCF header is meant to precede requests that are due immediately and is also used for responses, acknowledge and notifications.

Within the stream each request has a further identification, the **byte_bus_id**, to be used by the RC Server for identifying the endpoint the request is targeted to.

Requests, responses, acknowledge and notifications are formatted in either generic byte bus messages (ACF_GBB) or abbreviated byte bus messages (ACF_ABB). These are explained per type of endpoint in the respective chapters in this specification.

10.2 Networking

An RC Server will receive one or more streams according to its configuration. With a configurable look-up table, the RC Server decides based on the **stream_id** and **byte_bus_id** in each request to which of its endpoints the request is to be dispatched.

The endpoint will create responses, acknowledges and notifications using the **byte_bus_id** that was included in the request, hereby responses, acknowledge and notifications of one endpoint may have different targets, depending on the RC Client from which the request originated. RC Clients using the same endpoint may use different **byte_bus_ids**.

The RC Server collects responses, acknowledges and notifications from its endpoints and sends them in streams to RC Clients. Routing of the streams to the intended RC Clients is the responsibility of the network.

For each message to be sent, RC Clients and RC Servers select a (multicast) destination address depending on the identification of the stream.

11 USAGE OF IEEE1722 FOR RCP

11.1 Overview

IEEE1722 defines the control formats Generic Byte Bus (GBB) with **message_timestamp** and the Abbreviated Byte Bus (ABB) without timestamp.

The **message_timestamp** is interpreted as capture time stamp in responses and presentation time stamp in requests.

To transport one or more requests within the **acf_types** ABB or GBB they are preceded by a header building the AVTPDU.

The AVTPDU headers supported by RCP are the Time-Synchronous Control Format (TSCF) header and the Non Time-Synchronous Control Format (NTSCF) header.

The next figure shows a TSCF header in version 0 as it shall be used for RCP requests which shall be executed earliest at the included presentation time:

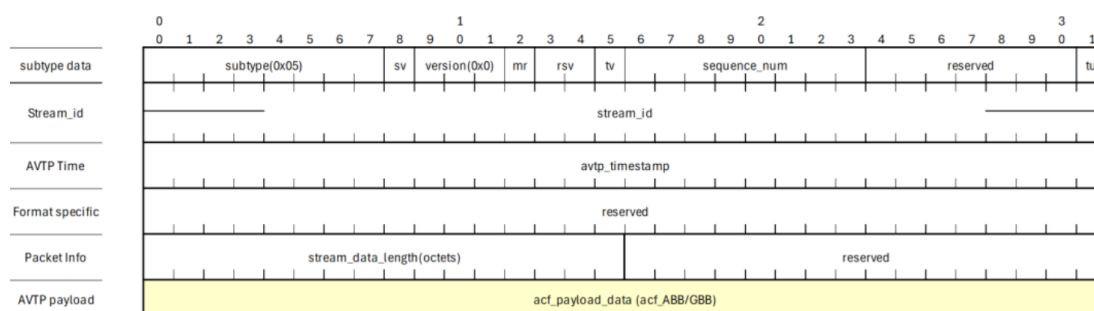


Figure 5 - TSCF-Header Version 0

The corresponding NTSCF header in version 0 as it shall be used for RCP requests which shall be executed as soon as possible as well as responses, acknowledge and notifications looks as follows:

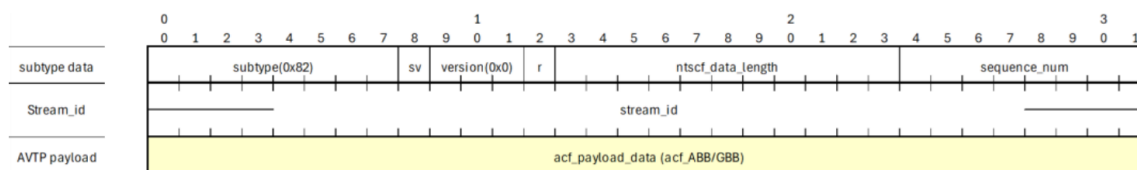


Figure 6 - NTSCF-Header Version 0

The RC Client sends requests to the RC Server by using either NTSCF or TSCF header and the message types ACF_ABB or ACF_GBB.

There are three basic request types with each having additional execution conditions timed and or WAIT:

1. Standard execution request
Standard request are executed as soon as possible if no further conditions are given, otherwise after the condition coming true.
2. Trigger based execution request
Execution is based on a trigger-event specified within the request. Trigger request can have an additional timed condition, which starts the trigger request earliest after the presentation-time has been reached.
3. Compound operation request
Execution is based on sequencer states and may be combined with timed or WAIT conditions.
4. Chain operation request
Execution is based on the predecessor request being completed

- 1 The RC Client uses ACF_ABB format for standard requests that are due for execution immediately, timed or conditional
- 2 WAIT. The WAIT condition is encoded in the event bits **evt[2:0]**.
- 3 The RC Client uses ACF_GBB format for trigger based or compound operation requests. Such conditions will be coded in
- 4 the **message_time_stamp** field. The **message_timestamp** valid (**mtv**) bit is cleared to indicate this alternative usage of
- 5 the bytes.
- 6 For a request with ACF_GBB format and mtv bit set to '1' the **message_timestamp** is interpreted as presentation time
- 7 and supersedes the **avtp_timestamp** if the ACF_GBB was received under TSCF header.
- 8 The next figure shows the structure of the ACF_GBB:

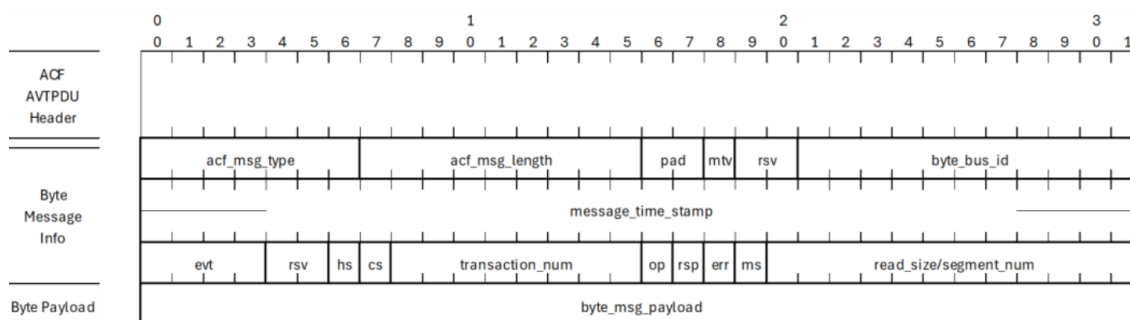


Figure 7 - ACF_GBB (ACF_ABB omits the "message_timestamp")

- 11 Explanation of the ACF_GBB and ACF_ABB fields:

Table: ACF-fields explanation

	In Requests from Client	In Ack/Response/Notification from Server
acf_msg_type	0x0E (ABB) 0x0D (GBB) (with timestamp or for trigger and compound request)	0x0E (ABB) 0x0D (GBB) (with timestamp)
acf_msg_length	size of "byte_msg_payload" and ACF header in quadlets	size of "byte_msg_payload" and ACF header in quadlets
pad (padding length)	Number of padding bytes	Number of padding bytes
mtv (message timestamp valid)	0: in ACF_ABB or for Trigger / Compound request 1: in ACF_GBB for presentation timestamp	0 for ACF_ABB 1 for ACF_GBB with capture timestamp
rsv (reserved)	0	0
byte_bus_id	Identification of byte_bus	Identification of byte_bus
evt (event)	evt[3] = 1 acknowledge requested evt[3] = 0 no acknowledge requested evt[2] = 1 request is a wait condition evt[2] = 0 request is no wait condition evt[1:0] endpoint specific usage	evt[3] = 1: acknowledge evt[3] = 0: response or notification evt[2] = 1: response after WAIT condition became true evt[2] = 0: response to unconditional request evt[1] not used evt[0] = 1: more responses with same

		transaction_num to come evt[0] = 0: response or ack
hs (hold stop)	depends on endpoint	depends on endpoint
cs (conditional start)	depends on endpoint	depends on endpoint
transaction_num	Number assigned by RC Client	Response/acknowledge: Same as in request. Notification: Strict monotonous increasing number. start at zero after power-on
op (operation)	0 = read request response with read data expected 1 = write request no data response expected only execution status EP executes on byte_msg_payload in any case!	Equal to op in request 0 = response to a read request, byte_msg_payload contains response data result (e.g. SPI received data or register content read) 1 = response to a write request, if err bit not set: no byte_msg_payload if err-bit set: error code in byte_msg_payload
rsp (response)	0 = identifies a request 1 not used by client	0 = not used by server 1 = identifies an acknowledge, response or notification
err	0 – always, as per IEEE1722 standard	0 = no error 1 = error message
ms (more segments)	0 = no additional segment(s) 1 = additional segment(s) to come (the maximum transmission unit length might be limited to 64bytes (incl header))	0 = no additional segment 1 = additional segments to come (the maximum transmission unit length might be limited to 64 bytes (incl header))
read_size/ segment_number	for a read request the read_size gives the amount of data to be returned in bytes details IEEE1722	no read size, if value given it belongs to fragmentation details IEEE1722
byte_msg_payload	depends on endpoint	depends on endpoint

1

2

11.2 Time synchronization

An RC ~~PHY~~ Server has to synchronize an internal clock to a system clock to support time synchronous execution of requests within a RC System as well as time-stamping of responses or notifications. This can be done using IEEE802.1AS (gPTP).

The RC ~~PHY~~s Server clock becomes a secondary clock.

11.2.1 General Information

Within IEEE1722 there are two different types of timestamps which are used within the RCP protocol:

3. For the TSCF header the `avtp_timestamp` is used:
4. for header V0: $\text{avtp_timestamp} = (\text{AS_sec} \times 10^9 + \text{AS_ns}) \bmod 2^{32}$ where `AS_sec` the gPTP seconds field is and `AS_ns` the gPTP nanoseconds field (thus rolls over every 4 seconds)
5. for header V1: $\text{avtp_timestamp} = (\text{AS_sec} \times 10^9 + \text{AS_ns}) \bmod 2^{64}$ where `AS_sec` the gPTP seconds field is and `AS_ns` the gPTP nanoseconds field (thus rolls over every 584,9 years)
6. For capture time stamps in ACF_GBB reponse messages the `message_timestamp` format is used.
 $\text{message_timestamp} = (\text{AS_sec} \times 10^9 + \text{AS_ns}) \bmod 2^{64}$ where `AS_sec` the gPTP seconds field is and `AS_ns` the gPTP nanoseconds field (thus rolls over every 584,9 years)

11.2.2 Capture time stamps (`message_timestamp`)

Time stamps in the ACF_GBB type sent by the RC Server represent the time when information has been captured and are in `message_timestamp` field.

Whether time stamps shall be created, can be selected for responses and acknowledge separately individually per endpoint. When timestamping was not requested, then the ACF_ABB type format without time stamp shall be default for any data transmitted by the RC Server.

The timestamp of an acknowledge represents the point in time when the request has been accepted by the respective EP request queue for execution.

The timestamp of a response represents the point in time when the physical signal has been captured (e.g. ADC sample acquired, GPIO pin changed, SPI CSN asserted, ...)

11.2.3 Presentation time stamp (`avtp_timestamp`)

A presentation time stamp (`avtp_timestamp`) can be found in the TSCF header and is valid for all requests in ACF_ABB types under this header.

Presentation time stamps are the earliest point in time, when a message shall be executed by the addressed endpoint(s).

The RC server may reject the request, when the presentation time is too far in the future, which is implementation dependent.

TSCF headers will only be sent by RC Clients, never by the RC Servers. The RC Server always uses NTSCF header and only uses `message_timestamp` in ACF_GBB to convey a capture timestamp.

11.3 The usage of the op bit in requests

Within the RCP a request is always seen as a call to action and the op bit as defined in IEEE1722 as the "operations field" decides if a write or read request shall be executed.

A read response contains the read data in its **byte_msg_payload**. A write response with the err-bit set to 0 indicates a successful operation and has no payload. A write response with the **err-bit** set to 1 indicates an execution error and may contain an additional error code in the **byte_msg_payload**.

Based on this concept op=1 means write and no data response which e.g. as access to the config memory space represents a write operation. With op=0 a data response is expected which depends on the endpoint. E.g. SPI or I²C can receive data based on the execution of the request's payload message. The response then will include the received data. For config register map access on EP0 the response will include the content of the addressed register. In any case the requests payload will be executed.

So for a read request to the config register map via EP0 the payload shall only include the register address to be read and no data. Otherwise, the data will be written first and the response with the new data be generated.

The amount of data in the response is given by the **read_size** field in the request.

11.4 The usage of evt bits in requests

The RC Client can use the bits **evt[3:0]** to indicate special handling of the request and its payload data.

evt[3] in a request is used to indicate if the RC Client requests to send an acknowledge that the request has been received and is queued for execution.

If on the contrary, the request cannot be added to a queue, an error message shall be generated back.

7. Set to '1' an acknowledge shall be generated.

8. Set to '0' no acknowledge is requested.

9. The acknowledge message will be sent as ACF_ABB with the evt[3] also being set and transaction number copied from the original request, with no byte_msg_payload via the acknowledge queue of the requesting RC Client

evt[2] bit decides if the requests payload data shall be handled as WAIT condition (**evt[2]=1**) or as data (**evt[2]=0**) that an endpoint has to execute.

evt[1:0] are depending on the request type and the endpoint type. They describe different ways of handling the **byte_msg_payload**. For details see section "Endpoint specific usage of evt-bits and **byte_msg_payloads**" in the chapter "Endpoints".

If a message has a presentation timestamp either from the TSCF header or from the ACF_GBB header, the message is not considered for execution until the given point in time is reached. Other conditions will be checked only when the specified time is reached.

The following table summarizes the handling of the event bits:

Table: Usage of evt-bits in requests

OP	EVT[3]	EVT[2]	EVT[1]	EVT[0]	
0	1	0	x	x	Normal read request with data response; evt[3] = 1 acknowledge requested evt[1:0] EP specific handling
0	0	0	x	x	Normal read request with data response; EVT[3] = 0 no acknowledge requested
0	x	1	x	x	Read request with data response and WAIT (evt[2] = 1): wait type endpoint specific evt[1:0] Standard request: WAIT for condition to execute request and send response data Standard with Time: if presentation time reached: WAIT for condition to execute request and send response Compound: Execute sequencer command after WAIT and send response Compound with Time: if presentation time reached: Execute sequencer command after WAIT and send response Trigger based: dropped by RC Server; no acknowledge if requested
1	1	x	x	x	write request; evt[3] = 1 acknowledge requested
1	0	x	x	x	write request; evt[3] = 0 no acknowledge requested
1	x	0	x	x	write request usage of byte_pay_load data endpoint specific depending on evt1:0]

1	x	1	x	x	write request and WAIT (evt[2] = 1) wait type endpoint specific evt[1:0] Only allowed for Compound request. Sequencer set to next_state after WAIT and no response will be send; acknowledge will be send after queuing if requested for all other request types the request will be dropped and no acknowledge will be send if requested
---	---	---	---	---	---

11.4.1 Request specific usage of evt-bit and op-bit

The following table represents the general request handling based on the type of request and the **evtbits**:

Request type	Header	op	evt[2]	evt[1]	evt[0]		Response	byte_msg_payload
STANDARD	NTSCF	0	0	0	0	READ request	data read	request cmd data
				0	1	READ request EP specific SPI EP: select cfg 1	data read	EP specific data handling
				1	0	READ request EP specific SPI EP: select cfg 2	data read	EP specific data handling
				1	1	READ request EP specific SPI EP: select cfg 3	data read	EP specific data handling
			1	x	x	WAIT with data response evt[1:0]: EP specific wait condition	data read	WAIT condition
		1	0	0	0	normal WRITE dropped for ADC, PWM_IN SPI EP: select cfg 0	execution status	write data
				0	1	EP specific WRITE dropped for ADC, PWM_IN SPI EP: select cfg 1	execution status	EP specific data handling
				1	0	EP specific WRITE dropped for ADC, PWM_IN SPI EP: select cfg 2	execution status	EP specific data handling
				1	1	EP specific WRITE dropped for ADC, PWM_IN SPI EP: select cfg 3	execution status	EP specific data handling
			1	x	x	dropped by RC Server	execution status	ignored

Table: Usage of EVT-bits in STANDARD NTSCF requests

STANDARD with presentation time	TSCF	0	0	0	TIMED READ request SPI EP: select cfg 0	data read	request cmd data
				1	TIMED EP specific READ SPI EP: select cfg 1	data read	EP specific data handling
			1	0	TIMED EP specific READ SPI EP: select cfg 2	data read	EP specific data handling
				1	TIMED EP specific READ SPI EP: select cfg 3	data read	EP specific data handling
			1	x	TIMED WAIT evt[1:0]: EP specific wait condition	data read	WAIT condition
		1	0	0	TIMED normal WRITE SPI EP: select cfg 0	execution status	request cmd data
				1	TIMED EP specific WRITE SPI EP: select cfg 1	execution status	EP specific data handling
			1	0	TIMED EP specific WRITE SPI EP: select cfg 2	execution status	EP specific data handling
				1	TIMED EP specific WRITE SPI EP: select cfg 3	execution status	EP specific data handling
			1	x	dropped by RC Server	execution status	ignored

Table: Usage of EVT-bits in STANDARD TSCF requests

COMPOUND	for TSCF exec request after presentation time reached	0	0	0	COMPOUND READ REQ SPI EP: select cfg 0	data read	request cmd data
			0	1	COMP D READ EP specific SPI EP: select cfg 1	data read	EP specific data handling
			1	0	COMP D READ EP specific SPI EP: select cfg 2	data read	EP specific data handling
			1	1	COMP D READ EP specific SPI EP: select cfg 3	data read	EP specific data handling
		1	x	x	COMP D READ with WAIT evt[1:0]: EP specific wait condition	data read	WAIT condition
	for NTSCF normal execution	0	0	0	COMP D WRITE request dropped for ADC, PWM_IN SPI EP: select cfg 0	execution status	request cmd data
			0	1	COMP D WRITE EP specific dropped for ADC, PWM_IN SPI EP: select cfg 1	execution status	EP specific data handling
			1	0	COMP D WRITE EP specific dropped for ADC, PWM_IN SPI EP: select cfg 2	execution status	EP specific data handling
			1	1	COMP D WRITE EP specific dropped for ADC, PWM_IN SPI EP: select cfg 3	execution status	EP specific data handling
		1	x	x	COMPOUND with WAIT evt[1:0]: EP specific wait condition	NO	WAIT condition

Table: Usage of EVT-bits in COMPOUND requests

TRIGGER	for TSCF exec request after presentation time reached	0	0	0	TRIGGER READ request SPI EP: select cfg 0	data read	request cmd data
			0	1	TRIGGER READ EP specific SPI EP: select cfg 1	data read	EP specific data handling
			1	0	TRIGGER READ EP specific SPI EP: select cfg 2	data read	EP specific data handling
			1	1	TRIGGER READ EP specific SPI EP: select cfg 3	data read	EP specific data handling
		1	x	x	dropped by RC Server	execution status	ignored
	for NTSCF normal execution	1	0	0	TRIGGER WRITE request dropped for ADC, PWM_IN SPI EP: select cfg 0	execution status	request cmd data
			0	1	TRIGGER WRITE EP specific dropped for ADC, PWM_IN SPI EP: select cfg 1	execution status	EP specific data handling
			1	0	TRIGGER WRITE EP specific dropped for ADC, PWM_IN SPI EP: select cfg 2	execution status	EP specific data handling
			1	1	TRIGGER WRITE EP specific dropped for ADC, PWM_IN SPI EP: select cfg 2	execution status	EP specific data handling
		1	x	x	dropped by RC Server	execution status	ignored

Table: Usage of EVT-bits in TRIGGER requests

11.5

Paragraph removed

11.6 The usage of the byte_msg_payload in requests

Each endpoint shall handle the **byte_msg_payload** in dependence of the **evt** bits contained in a request. The usage of the **byte_msg_payload** and the **evt** bits is described for each type of endpoint individually.

11.7 Compound operations – Sequences based request execution

Handling of compound operation request is an optional function for RC PHY Server implementations. In case this is not supported, requests to register a compound operation shall be dropped and a response send with **err-bit** set and error code **UNSUPPORTED_CMD**.

Compound operations can be used to build sequences of requests to be executed based on the state of a sequencer. Sequencers are simply a state register which stores the current state of the sequencer. Compound requests link to a specific sequencer via the **Sequencer_Nr** given in the request. An execution is started when the sequencers state matches the **start_state** given in the request. Additionally, the compound request contains the state (**next_state**) the sequencer shall enter after the sequencer operation has been executed.

If **next_state** is “0” then the sequencer remains in the current state. If **start_state** is “0” then the request shall be processed in any state (e.g. interesting for requests with **evt[2]=1**) unless the sequencer is disabled (equals “0”) which will prohibit the execution of the compound request. After power-on/reset all sequencers are in state 1.

The state of the sequencers can be read out via the RC Server. Also enabling, disabling, as well as setting the sequencer state is controlled via the RC Server.

ACF_GBB with **mtv=0** and specific usage of the **message_timestamp** field is a special case on top of IEEE1722, which defines that the message_timestamp is to be ignored in case **mtv=0**. In case of remote control, it is not ignored but has an alternative meaning.

The compound operations are configured using ACF_GBB having **mtv=0** (time stamp not valid) and using the bytes of the **message_timestamp** as defined below.

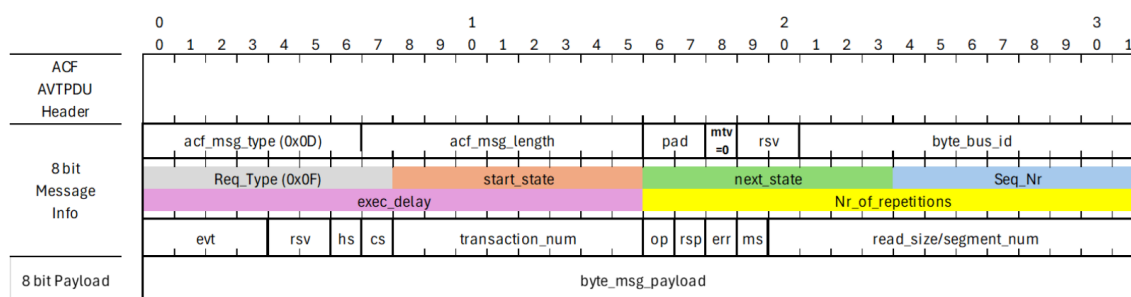


Figure 8 - ACF_GBB for COMPOUND request

Explanation of the fields:

Req_Type	0x0F or 0x8F for COMPOUND request
Start_state	Defines the state the addressed sequencer has to reach to execute this request, a zero as start_state will execute the request in any state (mostly used to start a sequence based on a WAIT condition)
Next_state	state the sequencer will be set to after the request has been executed, a zero as next_state leaves the sequencer state unchanged
Seq_Nr	addresses the sequencer used by this request
Exec_delay	delay time between the sequencer reaching the state defined in start_state and the start of the execution of the request in clock cycles
Nr_of_repetitions	number of times the request shall be repeated, if the field has a zero at the end of the execution, the request will be removed from the queue, otherwise decreased by one. If set to 0xFF_FF it indicates infinite repetition and no decrease is done at end of execution

The **Req_Type** shall be 0x0F or 0x8F. Compound operations with 0x0F will be cleared in case of a watchdog overflow of the RC Client that registered this request, while those with 0x8F remain active after watch-dog overflow. If such “compound operation” is received under a TSFC header, then it is executed for the first time earliest at the time of the presentation timestamp.

- 1 The number of sequencers and states per sequencer are limited to 256 by this definition. An RC ~~PHY~~ Server
2 implementation may support only a lower number of sequencers.
- 3 The maximum configurable number of repetitions is limited by this definition to 65534 (if not set to 65535 which means
4 unlimited repetitions). An RC ~~PHY~~ Server implementation may support only a lower number of repetitions.
- 5 **Evt[3]** can be used by the client to request an acknowledge to get confirmation that the compound request was
6 registered.
- 7

11.8 Triggered execution of requests

Each endpoint may generate one or more trigger events. These are described separately for each endpoint in the respective chapter. A trigger event is generated once the execution of a request is completed.

Each endpoint may also be configured to execute a request upon receiving a selected trigger event. The client issues a request for this purpose in the following format:

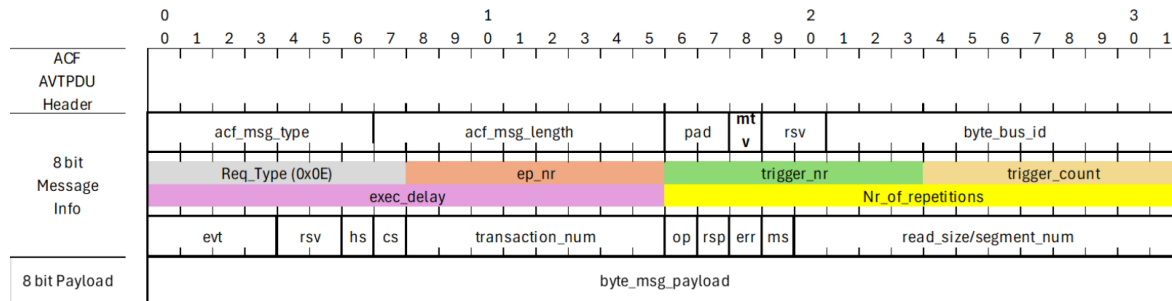


Figure 9 - ACF_GBB for TRIGGER request

Like for the compound operations the ACF_GBB with **mtv**=0 is used. **evt**[2] shall be 0 as the combination of a trigger and a WAIT condition is not senseful. If it is set the request will be dropped and a response send with **err-bit** set and error code **UNSUPPORTED_CMD**.

The Req_Type field for a TRIGGER request shall be 0x0E or 0x8E. The latter marks it as a safety relevant case to be executed under safe state. Trigger requests with 0x0E will be cleared in case of a watch-dog overflow of the client that registered this request, while those with 0x8E remain active after watch-dog overflow.

Explanation of the fields:

Req_Type	0x8E or 0x0E for TRIGGER request
Ep_nr	defines the endpoint which issues the trigger to be used for this request
trigger_nr	defines the trigger number of the endpoint addressed by ep_nr
trigger_count	defines how many triggers shall occur before the request is executed. e.g. '0' results in execution on every trigger while a '3' will cause an execution on every fourth occurrence.
exec_delay	delay in base_clock cycles of the endpoint for the request execution after the trigger occurred the as many times as specified by trigger_count
Nr_of_repetitions	number of times the request shall be repeated, if the field has a zero at the end of the execution, the request will be removed from the queue, otherwise decreased by one. If set to 0xFF_FF it indicates infinite repetition and no decrease is done at end of execution

The maximum configurable number of repetitions is limited by this definition to 65534 (if not set to 65535 which means unlimited repetitions). An RC ~~PHY~~ Server implementation may support only a lower number of repetitions.

If such “trigger operation” is received under a TSFC header, then this trigger operation shall be executed for the first time at the time of the **avtp_timestamp** at the earliest.

An endpoint may use its own trigger. With this and using the trigger delay a periodic execution of a defined request with a defined cycle time can be realized. Once the trigger execution has been requested by sending the correct ACF_GBB a first execution can be triggered by a request to the respective endpoint. To stop the execution the trigger request has to be cancelled.

E.g. an ADC is set-up to trigger itself with a trigger delay set to 1 ms and endless repetitions, then a single request to the ADC endpoint starts a cyclic reading with a cycle time of 1 ms. To stop that the request needs to be canceled.

In case of a trigger request that leads to an endless cyclic execution, a user may consider to limit the data on the communication network by setting `op=1` to not get a response on any execution, but use an additional compound operation with `evt[2]=1` in order to get a notification when certain defined conditions are met.

11.9 Chain operation (Optional feature)

If a single AVTPDU contains more than one request (**acf_types**) the execution of the request can be forced to be operated in a chain. A chain always starts with an arbitrary request, the following requests can then be chained via the chain request type:

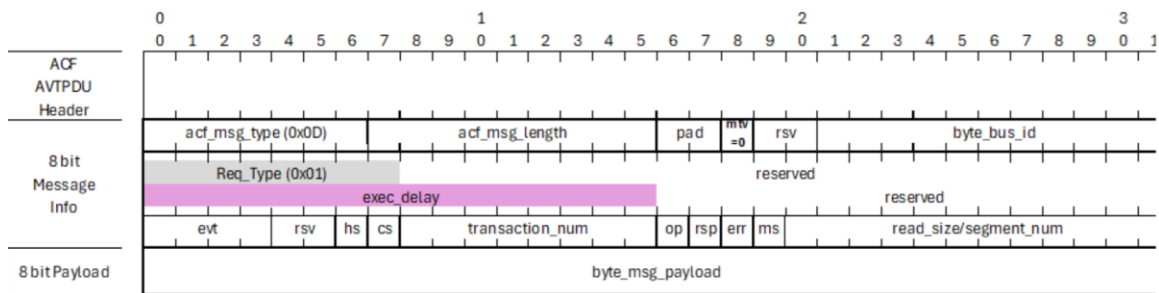


Figure 10 - Request chain

By this method two or more requests can be chained. If e.g. the first request had a WAIT condition the whole chain will only be started after the WAIT condition came true, as the second and following requests always wait on the predecessor request to finish its execution.

If the first request in an AVTPDU is a CHAIN request, the request will be ignored and an error response given with CHAIN_error, as there is no predecessor to chain to. If the first request in a chain has a repetition count, then the whole chain will be repeated in execution based on the first commands repetition count.

Alternative: The first request in a chain of requests shall always be a STANDARD request.

11.10 Cancellation of requests

There are three options to cancel requests:

1. Clear all request from a queue from an EP request queue

In this case the request type (first byte in the message_timestamp field) is 0x05

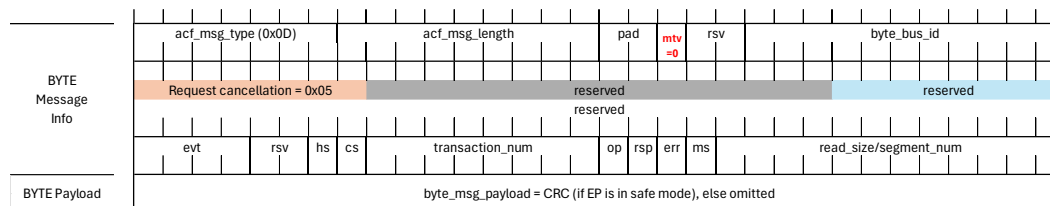


Figure 11 – Cancellation request

2. Clear all non-safestate requests from an EP request queue

In this case the request type (first byte in the message_timestamp field) is 0x06

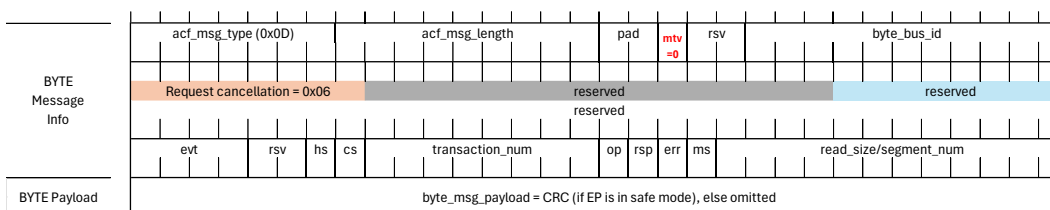


Figure 12 – Cancellation request

3. Clear one single request from an EP request queue

In this case the request type (first byte in the message_timestamp field) is 0x07 followed by the transaction_num of the request to be cancelled.

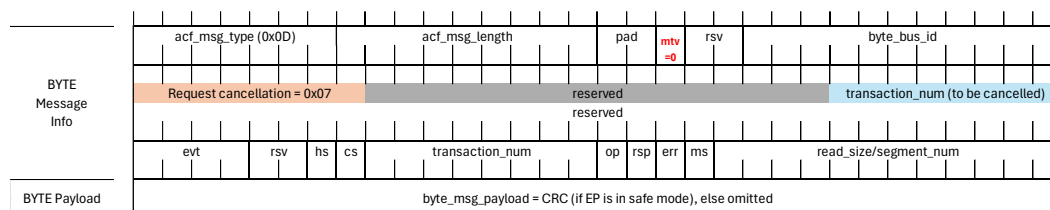


Figure 13 – Cancellation request

11.11 Configuration of endpoints

The configuration of endpoints can be changed by accessing the “EP_func” section of the respective EP via the RC Servers (EP0) register map or directly without the need to determined the base address of the respective EP in the register map.

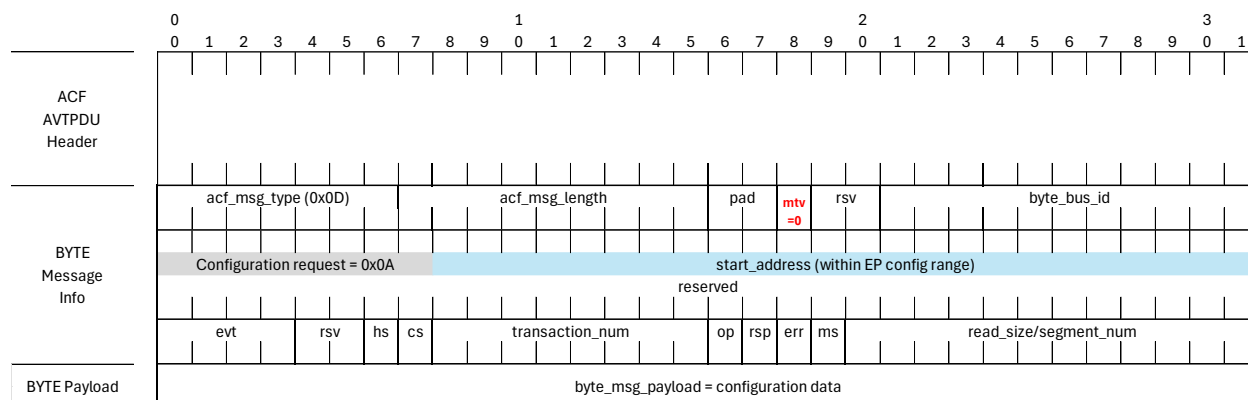
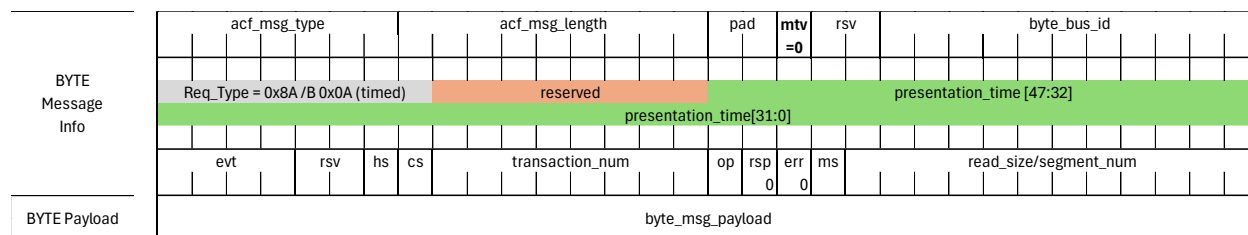


Figure 14 – configuration request

This configuration request format allows to access the EP_functional configuration. The data of the byte_msg_payload from a write request is written into the section of the EP that is addressed by the byte_bus_id. At start_address = 0x00 the EP_LEN parameter of the respective EP can be found as described for the EPs. Any byte_msg_payload for which the length plus the start_address results in a value larger than the EP_LEN, is to be ignored.

11.12 Configuration of endpoints

The RC Client may want to have a request being executed at a predefined point in time. For this purpose, requests can be sent under TSCF header, where a avtp_timestamp is included, that serves as a presentation time. A second option is to convey the presentation time in the message_timestamp field of an ACF_GBB. In this case the presentation time is given in 6 bytes, while the first byte is either 0x0A or 0x8A. 0x8A is meant for safety related execution.



11.13 Priorities in execution

Each endpoint receives its request via its dedicated queue.

Conditional execution may depend on whether a sequencer state matches, a trigger is valid, a timed request is due, or a WAIT becomes due.

Whenever an Endpoint is able to start a new execution, it shall process all entries in the queue in the order they have been put into that queue and execute the first one it finds due for execution.

„In case two or more requests are due for execution then the priorities are as follows

1. Requests for cancellation
2. triggered
3. timed
4. compound
5. wait
6. chained
7. standard

When two or more requests with the same priority are due then the execution shall happen in the order in that the requests arrived in the queue.

Hereby the conditions for execution are:

1. if it is a STANDARD request without conditions execute and remove from queue.
2. if for a TIMED STANDARD request the scheduled execution time has been reached, execute and remove from queue.
3. if a STANDARD or TIMED STANDARD request belongs to an EP group request, the serial execution is enabled and the actual EP is not the first in group, it shall wait for the predecessor EP in the group to finalize execution before its own execution starts. The EP stays anyhow busy until the last EP in the group request has finalized execution and then starts sending its data to the responder queue. Afterwards the request is removed from the queue.
4. if for a STANDARD WAIT request the condition is true, execute request and remove from queue.
5. if for a TIMED STANDARD WAIT request the presentation time has been reached and the WAIT condition is now true, execute the request and remove from queue.
6. if for a TRIGGER request the number of triggers specified has been received and the specified delay time after receiving the last trigger has been passed, execute the request and if number_of_repetitions is zero remove the request from the queue, otherwise decrease the number of repetitions by one if not 0xFFFF.
7. for a TIMED TRIGGER request the assessment of the trigger conditions only starts after the presentation-time has been reached. Trigger handling is identical to normal TRIGGER request. If the TRIGGER request is repetitive, the presentation-time is only used for the first execution and ignored for all subsequent ones.
8. if for a COMPOUND request the delay time after reaching the sequencer start_state has been passed, execute the request and then set the sequencer according to next_state if the sequencer state still equals the start_state. If number_of_repetitions is zero remove from queue, otherwise decrease number of repetitions by one if not 0xFFFF.
9. for a TIMED COMPOUND request the assessment of the sequencer state only starts after the presentation-time has been reached. Sequencer handling is identical to normal COMPOUND request. If the COMPOUND request is repetitive, the presentation-time is only used for the first execution and ignored for all subsequent ones.
10. for a COMPOUND WAIT request the assessment of the WAIT condition is only performed in the specified start state of the sequencer.
11. for a TIMED COMPOUND WAIT the assessment of the WAIT condition only starts after the presentation-time has been reached. WAIT condition and sequencer handling is then the same as for the COMPOUND WAIT request. If the TIMED COMPOUND WAIT request is repetitive, the presentation-time is only used for the first execution and ignored

- 1 for all subsequent ones. Anyhow as in the COMPOUND WAIT request the WAIT condition needs to become true in
- 2 each execution cycle.
- 3 After execution or no request was due for execution the processing of the queue is started again. Specially for trigger
- 4 requests the fulfillment of the trigger condition and start of the trigger delay time shall be independent of the idle state of
- 5 the endpoint.
- 6 The number of requests which can be stored in the queue is an RC Server configuration parameter and is limited by the
- 7 capabilities of the RC ~~PHY~~ Server. When a queue is full, newly arriving requests are dropped and a response with the
- 8 corresponding error code generated. It is the duty of the RC Client to avoid such overflow.
- 9

12 RC SERVER

12.1 Overview

The RC Server shall handle requests from one or multiple RC Client(s) which are addressed to its Endpoints. Responses and notifications received from the Endpoints will be handled by the RC Responder.

The RC Server shall provide access to its own configuration and the configuration of all its Endpoints, as well as the RC Responder. The configuration is stored locally in the RC Server's configuration memory.

Discovery of RC Servers means reading the RC Server's configuration memory. The configuration is stored as list of parameters. Access to these parameters depends on the lifecycle state of the RC PHY Server (see next chapter). All configuration parameters are writable in HW_unconfigured state. Some parameters are for information only and as such fixed by nature as described in the RC PHY's Server's datasheet (e.g. number and type of interfaces) or as described in this specification. For safety configuration data can be locked to be read only via the RC Servers configuration.

The RC Server shall provide access to status and debug information from its Endpoints and itself.

12.2 Lifecycle

The lifecycle of the remote-control system components describes phases of their usage.

12.2.1 Lifecycle of the RC Phy

The life cycle of the RC PHY starts at the production at the semiconductor maker. At the end of the RC PHY production the state is "HW_unconfigured". In this state the RC PHY does not contain application specific configuration. However, the number of supported endpoints and related physical interfaces are fixed (as by the datasheet).

The RC Server is operating and can reply to network queries to discover and enumerate the RC PHY, endpoints and interface properties. Endpoints may have a product specific default behavior as defined in the datasheet.

Depending on the capabilities of the RC PHY implementation (e.g. availability of Non-Volatile Memory) the RC PHY can be tagged in different configuration states. The state will be also reported during discovery.

4. HW_unconfigured: The RC PHY is completely unconfigured.
5. HW_configured: The IO-Pin mapping to endpoints and the usage of endpoints is configured.
6. RCP_configured: additionally to HW_configured state also all necessary RC Client, responder queues and EP functional configurations are done.

Only in the RCP_configured state the RC Server will accept requests to configured endpoints. In the other states only Standard unconditional requests are accepted for endpoint 0.

As mentioned depending on the capabilities of the RC PHY part or all of the configuration registers content may be stored in Non-Volatile-Memory, reducing the effort of the RC Client for configuration after power-on and discovery.

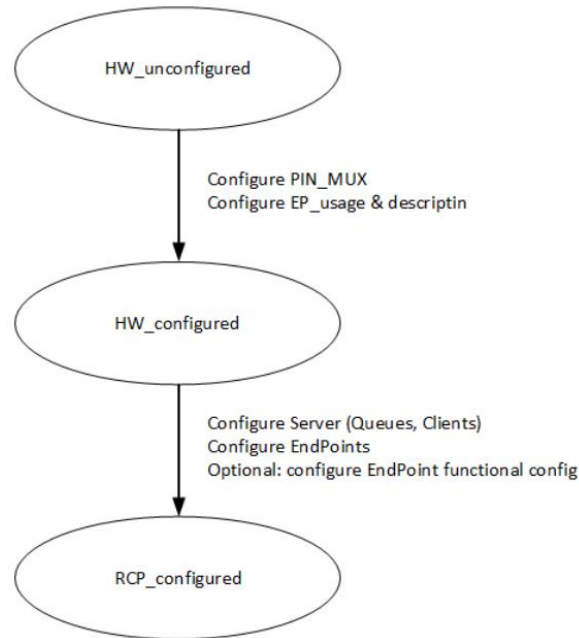


Figure 15 - RC PHY lifecycle states

Independent of the RC PHY's life cycle state it will answer to discovery requests within the remote control system. During discovery the RC Server presents its state and capabilities. The RC Client can configure or re-configure the endpoint's functional parameters as needed for the applications. It is possible to bind different endpoints to different RC Clients via their **stream_IDs**. If all necessary parameters to accept requests (Clients, Queues, Endpoints, ...) are configured the RC_PHY can be set to RCP_configured state by the RC Client.

After a power cycle or restart depending on the RC PHY's implementation all of the configuration information can be read from Non-Volatile-Memory included in or attached to the RC PHY and can be used immediately without a new discovery in the remote control system.

The RC Client is responsible for setting the LC_state of the RC Server.

12.2.2 Lifecycle of the Remote-Control Framework

The RCP Framework's lifecycle starts when the RC Client and RC Server are first connected to the same network during vehicle assembly. The RC Server may already be in the RCP_configured state. The RC framework is in the "discovery state".

In this state the RC Client on the controller side can discover all RC Servers in the network and their properties, if not a priori known. The RC Client can permanently store all discovered RC Servers and their properties. When the RC Client has acquired the knowledge about all RC Servers and re-configured them, if necessary, then the "normal operation state" is of the RC framework reached. The RC Client is then able to provide the device driver with a means to access the (remote) interfaces. The "normal operation state" is typically used in-field.

The framework can re-enter "discovery state" if needed, e.g. during repair of the vehicle, or when RC Servers are added, removed, or changed.

12.3 Power-On / Wake-Up / Start-Up behavior

There are two types of start-up: a cold start and a hot start.

A cold start appears after power on of the RC PHY, where the RC PHY needs first to either load its configuration from NV-memory or needs to be configured via the network.

A hot-start appears after sleep or low power mode when a wake-up is triggered. In hot-start the configuration does not need to be redone, as it shall be maintained during low-power/sleep.

Hot-start procedure:

The RC Server enables the network interface and sends a wake-up signal via the network in case the network is not operational yet. Once the network is available the RC Server sends status information (the reason why being active / wake-up source) repeatedly. An RC Client must acknowledge this notification to end it. The generation of this repeated message may be disabled in the WakeUp endpoint configuration.

12.4 Sleep behavior

The RC Client that is allowed to access the RC Server endpoint can request the entire RC PHY to enter sleep mode.

In sleep mode the power consumption shall be reduced to a minimum and only the wake-up control shall be active.

If an event defined as wake-up source in the wake-up endpoint occurs during the processing of the sleep-request, the request is not executed and the RC PHY does not enter sleep mode.

If the sleep request is executed, each Endpoint will complete an ongoing request execution (e.g. an SPI endpoint will send the remaining bytes of the current transfer) and then enter its endpoint specific idle state (e.g. PWM_OUT endpoint stops and goes into idle). As soon as all endpoints are in idle state the device enters sleep mode.

12.5 Discovery

A discovery request is a request to the RC Server Endpoint with **stream_id**'s unique identifier = 0 and **byte_bus_id**=0 and **transaction_num**=0. The request needs to have **op**=0 (response requested), the **evt** bits all set to 0 and no data payload.

A discovery request will normally be broadcasted by an RC Client to detect all RC Servers on the network at once.

The RC Server will create a response based on the discovery request a response to the RC Server with its server config table as payload content.

For each discovered RC Server the RC Client can read the register map in full or partially and maintain a table of relevant discovered RC Servers and their endpoints. Whether a discovered RC Server is relevant for the RC Client or not may depend on the properties of the RC Server or its Endpoints.

The RC Client can retrieve information from relevant Endpoints, whereby the relevance depends on the RC Controller's application. After discovering RC Servers and their Endpoints, the RC Client can configure them appropriate to control the endpoints.

In case discoverable information, i.e. the RC Server's configuration, is known a-priori on the RC Client side of the RC System, then a discovery process does not need to be started.

The discoverable information can be stored permanently in the RC Controller for immediate availability at next power-on.

12.6 Configuration

Access to the configuration and status information is per ABB or GBB messages. The usage is explained in the endpoint section for the endpoint type RC Server, since the RC Server exposes itself as "endpoint0" (EP0). As long as the RC PHY is not in RCP_configured state only access to EP0 is possible and only unconditional STANDARD requests are allowed as all other request types need a full configured RCP_PHY (e.g. request queues, responder queues, Client_EP_ID mapping, etc.). In this state the requests from the RC Server are directly sent to endpoint 0. Other valid requests to EP0 will be rejected with an error response with error code REQUEST_REJECTED. Requests to other EPs will be dropped until the RC PHY is in RCP_configured state.

The content of the configuration and status information is given in this chapter.

12.6.1 Configuration in RC Phy's hw_unconfigured state

The RC Phy is in HW_unconfigured state when it leaves the production of its vendor. The purpose of configuration in HW_unconfigured state is to instantiate Endpoints for given devices or functions on a printed-circuit board (PCB) schematic. The RC Client writes configuration data to the RC Server's configuration memory to perform this configuration. For this purpose, the RC Client may provide an API.

The RC Server allows to lock part of or the entire configuration to prevent unintended change of module or schematic specific configuration, i.e. entering the HW_configured state of the RC PHY's life cycle.

12.6.2 Configuration in RC Phys's hw_configured state

The purpose of configuration in HW_configured state is to configure or update the runtime parameters of Endpoints (and their related interfaces) during in-field usage. E.g. change the frequency of a PWM signal.

The RC Client can write (update) values of the RC Server's configuration memory and for this purpose may provide an API to the Device Driver and/or application. Some parameters may be locked and thus read only in RC PHY's HW_configured state.

Once the intended endpoint configuration parameters (e.g. SPI clock polarity) have been configured the RC Client may set the RC PHY's lock bits to prevent unintended change of configuration parameters.

12.6.3 Register map - General part

The RC Server register map is split into sections and contains a first part that is common for all (OA TC18) RC Servers, to allow an RC Client to identify that the read response comes from an (OA TC18) RC Server. This common part is in the following table:

Absolute address	Parameter	Type	R/W	Description
0x0000	OA_TC18_MAGIC	32 bit	R	OATC18 Magic number / a constant t.b.d by TC18 / this is to allow a Client to conclude that this is an RCP server
0x0004	Version	32 bit	R	OATC18 RC Protocol Version Supported
0x0008	VENDOR_ID	16 bit	R	OATC18 defined Vendor ID for manufacturer of RCP Device (vendor= silicon manufacturer)
0x000A	DEVICE_ID	16 bit	R	Vendor specific device ID (allows in combination with VENDOR_ID to uniquely identify the device)
0x000C	EP_Count	16 bit	R	Number of implemented endpoints in this device
0x000E	Max_request_streams	8 bit	R	Maximum number of requests streams that can be used to access this RC Server
0x000f	Response_streams_used	8 bit	R	Maximum number of supported responder queues for communication between this RC Server and RC Clients
0x0010	Total_RespQueue_Mem	16 bit	R	Maximum queue memory in 32bit words
0x0012	Total_ReqQueue_Mem	16 bit	R	Maximum queue memory in 32bit words for EP request queues
0x0014	Nr_Sequencers	8 bit	R	0: Sequencer operation not supported 1..n number of available sequencers state registers
0x0015	Nr_WakeUp_Pins	8 bit	R	Defines the maximum number of Pins to be selected as wake up source
0x0016	Nr_EP_Groups	8 bit	R	Nr of maximum supported endpoint groups 0: EP groups are not supported
0x0017	Nr IO Pins	16 bit	R	Number of I/O Pins assignable (Details and restrictions may be in the Device specific datasheet)
0x0019	ROOT_CLIENT	8 bit	R/W	IDx in Client_Config of Client that is allowed to configure the RC Server 0: all RC Clients are allowed to write
0x001A	LC_State	8 bit	R/W	Life-cycle State: HW_unconfigured, HW_configured, RCP_configured
0x001B.0	HW_CONFIG_LOCK	1 bit	R/W	1: Prohibit write access to HW_Config
0x001B.1	Client_CONFIG_LOCK	1 bit	R/W	1: Prohibit write access to Client_Config
0x001B.2	Queue_CONFIG_LOCK	1 bit	R/W	1: Prohibit write access to Queue_Config
0x001B.3	EP_ID_CONFIG_LOCK	1 bit	R/W	1: Prohibit write access to EP_ID_Config
0x001B.4	EP_CONFIG_LOCK	1 bit	R/W	1: Prohibit write access to EP_Config
0x001C	Ptr_HW_config	16 bit	R	Pointer to HW_config register map See: 12.6.4 HW pin mapping configuration

0x001E	Ptr_Client_config	16 bit	R	Pointer to Client_config register map See: 12.6.5 Request stream configuration
0x0020	Ptr_Queue_config	16 bit	R	Pointer to Queue_config register map See: 12.6.8 Response stream configuration
0x0022	reserved	16 bit	R	
0x0024	Ptr_EP_config	16 bit	R	Pointer to EP_config register map See: 13.2 Generic part of endpoint register map
0x0026	Ptr_EP_ID_map	16 bit	R	Pointer to EP - 8 bit_bus_id mapping table See: 12.6.6 Endpoint ID and communication configuration
0x0028	Ptr_EP_Func_config	16 bit	R	Pointer to EP_FUNC_config register map See: 13.7.1.2 Server EP register access
0x002A	Ptr_EP_Status	16 bit	R	Pointer to EP_status register map See: 13.4 Endpoint status information
0x002C	Ptr_EP_Group_config	16 bit	R	Pointer to EP group table See: 12.6.7 Endpoint group configuration
0x002E	Ptr_Sequencer	16 bit	R	Pointer to Sequencer_config register map See: 12.6.9 Sequencer state registers
0x0030	Ptr_Network_config	16 bit	R	Pointer to Network_config register map See: 12.6.10 Network interface configuration
0x0032	Ptr_ptp_config	16 bit	R	Pointer to PTP_config register map 0: ptp is not supported See: 12.6.11 PTP configuration

Table: RC Server_config

The read only part of the register map further includes information on the RC PHY capabilities (e.g. number of concurrent RC Clients to be served, number of available endpoints, extended features, ...).

The ROOT_CLIENT selects out of the Client_config map the RC Client which is after configuration is finished still allowed to access all registers of the register map. All other RC Clients are only allowed then to access the Server_Config, HW configuration in read only mode or write data to their bound endpoints' functional configuration maps.

The HW_config register map contains information on the pin binding to endpoints as well as the pin properties, like Input, Output, IO, drive strength, The options here may be RC PHY specific.

The EP_config register map contains generic parameters common for all endpoints. The EP_FUNC_config contains the endpoint functional parameters which are specific to the endpoint type.

The register map further contains links to the network interface, and cyber security, which are described in detail in the respective sections in this document.

12.6.4 HW pin mapping configuration

As long as the device is in the HW_unconfigured state, there is no assignment of endpoint signals to physical IO-Pins of the RCP PHY. To make those assignments and also define the physical IO-Pin properties the HW_config register map needs to be configured.

This map contains for each used physical IO-pin an entry defining its mapping to an endpoint signal, as well as the properties of the IO-pin. The number of IO-pins that are available is a constant that can be read from the general part of the servers register map.

Relative Address	Parameter	Type	R/W	Description
IO_Pin 0				
0x0000	EP_Nr	8 bit	R/W	Endpoint Nr mapped to this IO
0x0001	EP_Pin_Nr	8 bit	R/W	Endpoint Pin Nr mapped to this IO
0x0002	DIR_Drv_Type	8 bit	R/W	Properties of the IO Pin (In, Out, Pull Up/Down ,...)
IO_Pin 1				
0x0003	EP_Nr	8 bit	R/W	Endpoint Nr mapped to this IO
0x0004	EP_Pin_Nr	8 bit	R/W	Endpoint Pin Nr mapped to this IO
0x0005	DIR_Drv_Type	8 bit	R/W	Properties of the IO Pin (In, Out, Pull Up/Down ,...)
IO_Pin 2				
0x0006	EP_Nr	8 bit	R/W	Endpoint Nr mapped to this IO
...				

Table: HW_config

The pin direction and drive type entry are split into 4 sections to decide on pull-up or -down resistor attachment, output state drive behavior and drive strength, as well as if in input only mode a schmitttrigger is applied or not. All outputs are always also an input.

Parameter	Bit	Description
Pull-Up	1:0	00: float 01: pull down 10: pull up
Output stage	3:2	00: Input 01: Open drain 10: Open source 11: Push pull
Drive strength	5:4	00: Input 01: Low drive 10: Medium drive 11: High drive
reserved	6	reserved reads 0
Schmitt-Trigger	7	0: no Schmitt-Trigger hysteresis applied 1: Schmitt-Trigger hysteresis applied

Table: IO-Pin properties

- 1 How many signals a specific endpoint has and the function of each signal needs to be read from the RC Phy's datasheet.
- 2 Since the pin configuration is a one-time event this is kept simple.
- 3 The mapping of the EP_Pin_Nr is specific per endpoint type and listed here:

EP_Type	EP_Pin_Nr	Description
GPIO	0	GPIO pin 0 to 31 according to EP capability
	...	
	31	
SPI	0	SPI_CLK
	1	SPI_CS
	2	SPI_COPI
	3	SPI_CIPO
I ² C	0	I2C_SCL
	1	I2C_SDA
UART	0	UART_TX
	1	UART_RX
	2	UART_RTS
	3	UART_CTS
LIN	0	LIN_TXD
	1	LIN_RXD
	2	LIN_NSLP
PWM_OUT	0	PWM_OUT positive phase
	1	PWM_OUTN inverted phase
PWM_IN	0	PWM_IN
ADC	0	ADC_IN
DAC	0	DAC_OUT
CAN	0	RXD
	1	TXD
ISELED	0	MI
	1	MO

Table: Endpoint Pin numbers

1

2 **12.6.5 Request stream configuration**

3 For the RC Server to accept requests the submitted **Stream_ID** needs to be known by the RC Server. Binding of the
 4 **Stream_IDs** to allowed RC Clients is done in the Client_config register map.

Relative address	Parameter	Type	Description	
Request stream 1				
0x0000	Stream_ID	64bit	ID of Request Stream 1	Default value is 0x0000000000000000 as consequence all streams are received
0x0008	WD_TimeOut	16 bit	WatchDog time out for this Client in clock tics	
0x000A.0	Enforce SEC	1 bit	0: stream is not blocked, when CRC check at EP fails (EP drops the request if CRC check fails) 1: stream is blocked until released, when CRC check at EP fails. Safe state will be entered	
0x000A.1	Enforce SEQ	1 bit	Requests are only queued for execution if sequence number in ACF type is increased	
0x000A.2	SEQ_SafeState_EN	1 bit	bring all endpoints to safety state if Sequence_Nr has no single increment	
0x000A.3	WatchDog_EN	1 bit	enable Watchdog for this Client	
0x000A.4	WD_SafeState_EN	1 bit	bring all endpoints for this Client to safe state if WD triggers	
0x000A.5	Q_OVFL_SafeState_EN	1 bit	bring all endpoints for this Client to safe state if request queue of one EP overflows	
0x000A.6	Safety_Measure	1 bit	0: as safe state set all IO Pins to high impedance 1: use sequencer based safety requests	
0x000A.7	WD_ClientInfo_EN	1 bit	send repetitive notification when in safe state	
0x000B	SafeState_Seq_Nr	8 bit	Sequencer Nr used for safety requests	
0x000C	SafeState_SeqState	8 bit	Sequencer state to be set to execute safety requests	
0x000D	ACK_stream_ID	8 bit	R/W	QUEUE ID points to the stream with which the endpoints used by this client send their acknowledge (0: no ack to be send)
0x000E	Response_stream_ID	8 bit	R/W	QUEUE ID points to the stream with which the endpoints used by this

				client send their responses (0: no ack to be send)
Request stream 2				
0x000F	Stream_ID	64bit	ID of Request Stream 2	
...	...	16 bit	...	

Table: Request_Stream_config

Each Request stream it is configurable which EP is addressed via which byte_bus_id. See next section.

For safety measures a strict monotonous increasing sequence number of the requests can be enforced. If a sequence error occurs a safe state may be enforced. The safe state to enter can be chosen via the Safety_Measure parameter. A similar mechanism can be additionally configured for the watchdog to ensure a continuous uninterrupted connection between the RC Client and its endpoints. Therefore the watchdog is reset with each received request from this RC Client.

If the watchdog exceeds its threshold again it can be selected if a safety measure shall be taken and if so which. If the safety measure to be taken is to run a safety request sequence via compound operations, a sequencer can be used for this and configured to trigger the execution of these safety requests.

12.6.6 Endpoint ID and communication configuration

The Server configuration defines to which streams the RC Server is listening as described in the Client_config. To address individual EPs for each Client the EPs mapped and their **byte_bus_ids** as well as their responder queues need to be defined.

The following configuration structure defines the client-endpoint-**byte_bus_id** combinations via which the requests are then mapped to the endpoints. As well as the responder queues for requests coming from a specific RC Client.

For each RC Client defined in the Client_config register map the list contains the response queues for the RC Client and a list entry with the EPs and the **byte_bus_ids** to be used; as well as endpoint groups per client. If an RC Client doesn't have EP groups the parameter Nr_of_EP_Groups is set to zero and the following parameters are all omitted. If for one RC Client the same **byte_bus_id** is assigned to more than one endpoint, only the first endpoint in the list will be addressed by a request to this **byte_bus_id**.

relative address	Parameter	Type	R/W	Description	default
0x0000	1_Request_Stream_Index	8bit	R/W	Index of request stream ID	1
0x0001	1_BBID	16bit	R/W	Byte_bus_id [11bit]	0
0x0003	1_EP_Nr	8bit	R/W	EP addressed by 1_BBID	0
0x0004	2_Request_Stream_Index		R/W	Index of request stream ID	0
0x0005	2_BBID		R/W	Byte_bus_id [11bit]	0
0x0007	2_EP_Nr		R/W	EP addressed by 2_BBID	0
0x0008	3_Request_Stream_Index		R/W		
	...				
	...				

Table: EP_ID_config (Request_stream & byte_bus_id mapping to Endpoints)

The default values allow to access EP0.

Note that one RC Client may use multiple streams to address the same endpoint, so will have multiple entries in the client_config. For safety reasons it is recommended that one endpoint is only used by one RC Client at a time.

The RC Client is responsible for configuring at least one stream-**byte_bus_id** combination for each endpoint. If multiple endpoints are using the same **byte_bus_id** inside one stream they shall be of the same type. To address different types of endpoints via a single **byte_bus_id** it is necessary to configure an EP_Group.

12.6.7 Endpoint group configuration

Endpoint groups are a set of endpoints within one **stream_id** which are combined in one **acf_type**. The execution of the requests to the endpoints within a group can be selected to be parallel or sequential. In any case a response shall contain the response data of each EP in the group in the order they are defined in the group.

If sequential execution is desired all EPs need to be idle before the first EPs request execution is started and all EPs stay after their request execution busy until the last EP in the group has finished its execution and all responses have been created.

For usage of endpoints in endpoint groups some restrictions apply:

1. all grouped endpoints need to be linked to the same Client_ID.
2. EPs with variable length payload (e.g. SPI) will generate within a response only the amount of data as in the request.
3. an EP group can only be accessed via STANDARD requests and supports only time condition.
4. The **read_size** field in the ACF header is the **read_size** over all endpoints in the group. Each fixed length EP generates one response quadlet, and EPs with variable length will generate data according to their request payload data length. Note: EPs returning less non quadlet aligned amount of data will be filled to the next quadlet boundary in the response

Configuration of endpoint groups is done within the EP_GROUP_config as shown below. The **byte_bus_id** mapping for an EP group access is done in the EP_ID_config map.

Relative address	Parameter	Type	R/W	Description
0x0000	Nr_of Groups	8 bit	R/W	Number of defined EP Groups
EP_Group 1				
0x0001.0	EXEC_SEQ	1 bit	R/W	0: Group EP requests can run in parallel 1: Group EP requests will be executed sequentially
0x0001.1	STOP_EXEC_ON_ERROR	1 bit	R/W	1: Sequential group request execution stopped on error of one EP execution 0: Execution of all requests continues independent off an individual error
0x0002	NrOfEPs	8 bit	R/W	number of EPs in EP group 1 Client 1
0x0003	EPNr	8 bit	R/W	EP number of first group member
0x0004	EPNr	8 bit	R/W	EP number of second group member
0x0005	...	8 bit	R/W	EP number of next group member until NrOfEPs
EP_Group 2				
0x0006.0	EXEC_SEQ	1 bit	R/W	0: Group EP requests can run in parallel 1: Group EP requests will be executed sequentially
0x0006.1	STOP_EXEC_ON_ERROR	1 bit	R/W	1: Sequential group request execution stopped on error of one EP execution 0: Execution of all requests continues independent off an error
0x0007	NrOfEPs	8 bit	R/W	number of EPs in EP group 2 Client 1
0x0008	EPNr	8 bit	R/W	EP number of first group member EPgrp 2
0x0009	EPNr	8 bit	R/W	EP number of second group member EPgrp 2
0x000A	...	8 bit	R/W	EP number of next group member until NrOfEPs

Table: EP_GROUP_config

- 1 An endpoint group allows for example the combination of a GPIO endpoint (e.g. endpoint nr 9) and a PWM_out endpoint
- 2 (e.g. endpoint nr 5). An RC Client can then send a combined ACF_type request. The endpoint group config for this
- 3 example would have NrOfEPs set to 2 and then 9 and 5 as byte values for the combined endpoints.
- 4 The first endpoint in that group uses the first quadlet of the **byte_msg_payload**, the second endpoint in that group is
- 5 using the second quadlet, etc.
- 6 For endpoints having a variable length of payload the first sixteen bits in the quadlets used give the length of the payload
- 7 in bytes. The next endpoint then again starts on a new quadlet.
- 8 An ACF_type access would then look as follows:

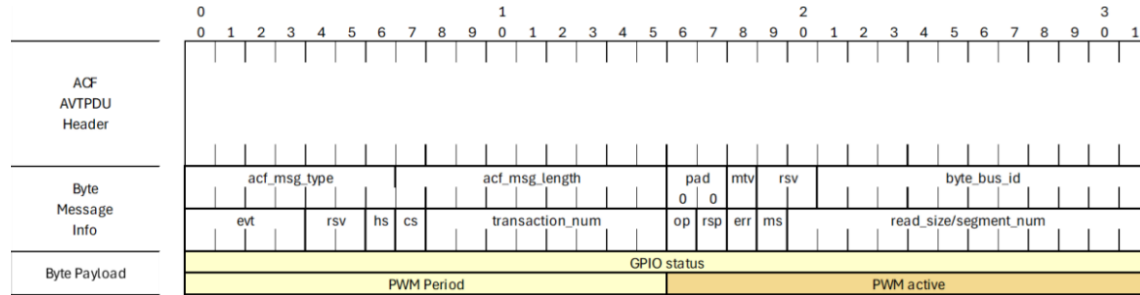


Figure 16 - ACF_type access

12.6.8 Response stream configuration

For each response or acknowledge stream the RC Server needs to reserve a transmit queue to collect responses and acknowledges from endpoints and the RC Server itself. The size of a queue is given by the parameter "queue_size".

The maximum length of an AVTPDU send by the RC Server shall be configurable to be able to avoid sending very long frames and is discoverable in the RC Server Info/Config register map. The Max_AVTPDUsize shall always be configured such that the final network frame does not exceed the maximum transmit unit size of the network.

In case an AVTPDU containing a single ACF_type would exceed the "Max_AVTPDUsize" fragmentation as supported in ACF_ABB and ACF_GBB by the ms-bit will be performed.

Once a queue is filled with an amount of quadlets that is equal or larger than given by "flush_on_count", the transmission of one or multiple AVTPDUs shall be initiated. Hereby only as much as fitting to the MAX_AVTPDUsize ACF_types will be included in a generated AVTPDU. Basically all ACF_types including the one which was exceeding the Flush_on_Count value will be transmitted, packed in a fitting number of AVTPDUs.

The transmission of an AVTPDU shall be initiated also in case the time since the last transmission from this queue is equal or greater than the Flush_time.

An AVTPDU shall be generated in this case even when the queue is empty to generate a heartbeat, so that the RC Client sees that the RC Server is alive.

The configuration is handled within the QUEUE_config of the RC Server configuration:

Relative address	Parameter	Type		Description	Default
Responder Queue 1					
0x0000	STREAM_UID	16 bit	R/W	[63:48] Unique stream ID of queue 1	0
0x0002	Max_AVTPDUsize	16 bit	R/W	Maximum length of an AVTPDU generated in quadlets	
0x0004	queue_size	16 bit	R/W	Queue 1 assigned memory in 32bit words	
0x0006	flush_on_count	16 bit	R/W	1: immediate 2 to Queue_Size in 32bit word entries (fixed length EPs have 3x32bit words wto timestamp and 5 with timestamp)	1
0x0008	Flush_time	16 bit	R/W	0: Flush only by count nr: time in µs	0
Responder Queue 2					
0x000A	STREAM_UID	16 bit	R/W	[63:48] Unique stream ID of queue 1	
...	...				

Table: RC Responder QUEUE_config

12.6.9 Sequencer state registers

To realize the execution of a chain of operations the RC Client can queue operations and their execution condition at endpoints. The execution condition is in any case the entrance of a given sequencer into a given state. After execution the sequencer is advanced to a defined next state.

Each sequencer state is represented by a value that is readable in the server's register map. Upon power-on reset all sequencer state values are set to "1".

How to register a request for operation for a chain of operations is explained in "Compound operations".

Each sequencer is dedicated to a specific RC Client and its bound endpoints. The implementation of one or more sequencers into an RC PHY is optional. If the feature is available and how many sequencers are available can be found in the respective part of the RC Server's register map. Manual access to the sequencer state is given via the SEQUENCER_config register map:

Relative address	Parameter	Type	R/W	Description	Default
Seq_1					
0x0000	Seq_state	8 bit	R/W	Recent state if sequencer no 1; if manually set to 0 then disabled	1
0x0001	Client_Nr	8 bit	R/W	refers the Client Nr allowed to access this sequencer	
Seq_2					
0x0002	Seq_state	8 bit	R/W	Recent state if sequencer no 2; if manually set to 0 then disabled	1
0x0003	Client_Nr	8 bit	R/W	refers the Client Nr allowed to access this sequencer	
Seq_3					
...					

Table: SEQUENCER_config

12.6.10 Network interface configuration

The network interface configuration depends on the actual network interface the RC PHY has.

Relative address	Parameter	Type	R/W	Description
0x0000	Network_interface_type	8 bit	R	0x00: 10BASE-T1S 0x01: 100BASE-T1
0x0001	Network_config_length	8 bit	R/W	Number of bytes describing the network interface

Table: Network

12.6.10.1 10BASE-T1S network configuration

Relative address	Parameter	Type	R/W	Description
	TO_length			
	IFG_length			
	1 st Response_streamID_index	8 bit	R/W	
	PLCA_slot	8 bit	R/W	PLCA slot to be used
	1 st DA	48bit	R/W	DA to be used, if 0x000000000000, then the SA of the received request stream that is linked to this response stream is to be used
	1 st VLAN_tag			

12.6.10.2 100BASE-T1 network configuration

<Not to be specified by Version 1.0>

1
2
3
4
5
6
7
8

12.6.11 PTP configuration

Here the relevant configuration parameters to set-up gPTP time synchronization are defined.

12.7 Communication handling

12.7.1 Network interfaces

The network interface needs network specific parameters e.g. :

7. In case the network interface is 10BASE-T1S one or more PLCA slots have to be configured in that the RC PHY is allowed to send.
8. If IEEE1722 is used on top of UDP/IP, IP-configuration might be necessary.
9. In case the network interface is CAN(FD/XL), specific configurations are to be made; e.g. bit rate settings.

12.7.2 Reception of frames

After correct reception of a frame on the RC PHY's network interface security information might need to be checked and removed if enabled (e.g. MACSEC).

If neither a gPTP nor a IEEE1722 frame is found the received frame shall be discarded.

Once an IEEE1722 AVTPDU has been extracted it is handed over to the RC Server. The RC Server checks the **stream_id** and if known to the RC Server extracts the contained requests and queues them in the addressed Endpoints queues.

~~Fragmentation as supported by ACF_GBB/ACF_ABB needs to be taken into account to get complete request.~~

The RC PHY shall be able to receive and handle frames with a minimum time between the frames/packets. E.g. for ethernet the minimum interpacket gap or for CAN the intermission time.

12.7.3 Handling of requests, responses, acknowledgements and notifications

12.7.3.1 Handling of requests

The RC Server handles requests handed over from the network interface.

In dependence on the **stream_id** and **byte_bus_id** the RC Server determines the endpoint that is addressed. If the lookup of the **byte_bus_id** in the context of the **stream_id** does not point to an Endpoint, the request is dropped without further notification.

For configuration purposes the RC Server itself is also treated as an endpoint. It may have within each configured **stream_id** an own **byte_bus_id**, which is zero as a default.

Requests are delivered by the RC Server to the queue of the addressed Endpoint(s) (see section "Priorities in execution").

The RC Responder will provide several responder queues that are assigned 1:1 to a **stream_id** (and with that to a destination which can also be a multi-cast destination).

Each Endpoint can be configured to which responder queue its responses, acknowledge or notification shall go in dependence of the stream from which the request was received.

An acknowledge is given if requested by the RC Client via setting **evt[3]=1** within the acf header as soon as the request has been queued in one of the addressed endpoint queues and therefore is scheduled for execution.

A response with pay load data read from the EP is given, if requested by **op=0** after the request has been executed.

A response with **err=0** and no payload is given after successful execution of an request with **op=1** as long as not configured to be suppressed within the EP_GENERIC_config. If an request could not be executed for any reason, a response with **err=1** is send and an **error_code** in the **byte_msg_payload** is given.

Response(s) and acknowledge(s) are associated to the requests by the **byte_bus_id** and transaction number.

If a request is received with **err=1** it shall cancel an existing request in the EP queue. The request to be canceled is identified by the RC Client it send it, the **byte_bus_id** and **transaction_num**. If the request can be canceled successfully the EP will send a response with **err=1** and error code REQUEST_CANCELED, for the canceled request no response is send. If the request was already under execution, the EP sends a response with **err=1** and error code REQUEST_NOT_FOUND. As the to be canceled request was already executing the EP will generate a normal response for it.

12.7.3.2 Handling of responses

The RC Responder will provide several responder queues that are assigned 1:1 to a **stream_id** (RC PHY MAC address + 16bit unique **stream_id** identifier and a destination address which can also be a multicast destination). Each RC Client has a configuration via which responder queue requests to Endpoint shall generate responses.

When the Endpoint is configured to timestamp responses, then the response will be in ACF_GBB, otherwise in ACF_ABB format.

The Endpoint provides responses associated to the requests identified by the transaction number of the request to the RC Responder. In case more than one response is to be generated (e.g. in case of a repetitive trigger or compound operation), all responses expect the last one, will have **evt[0]** set, so that the Client knows when the transaction number can be released.

12.7.3.3 Handling of acknowledge

An RC Client can request an acknowledge for the request send via setting **evt[3]=1**.

An acknowledge is given if requested as soon as the new request has been successfully queued for execution in the addressed endpoints queue.

If the request is a cancellation request (**err=1**) and an acknowledge is requested the acknowledge is only sent if the to be cancelled request was still in the endpoints queue and has been successfully removed.

If an AVTPDU frame sent by an RC Client contains multiple ACF_types (requests), for each ACF_type individually an acknowledge can be requested and will be sent. An acknowledge is send via the Client's configured responder queue to the RC Responder.

The acknowledge is associated to the request by the transaction number of the request.

12.7.3.4

Paragraph removed

12.7.3.5 Handling multiple requests in incoming messages

An RCP frame might be sent as multicast and therefore be received by multiple RC Servers.

An RCP frame may include multiple ACF-types (requests).

An RC Server shall support to handle multiple requests in one frame and check each of them individually if to be processed or not (**byte_bus_id/stream_id** match) and sort the requests into the corresponding request queues of the endpoints.

As the AVTPDU has either a TSCF or NTSCF header a mixture of requests with and without presentation time in a single frame is not possible. A presentation time from a TSCF header will be applied to all ACFtypes within the frame. The RC Server shall support the handling of multiple request types in one frame.

Request types may be:

10. Standard with or without WAIT

11. Trigger

12. Compound with or without WAIT under either TSCF or NTSCF header which decides then on a timed execution.

12.7.3.6 Handling of errors

Endpoints will generate error responses depending on execution errors or HW failures (e.g. an output pin could not be driven to the desired state) depending on the RC PHY` s capabilities.

Endpoints shall be configurable to timestamp error responses.

The error response shall contain the **byte_bus_id** and transaction number of the request. The error response shall contain a **byte_msg_payload** with an error code.

Error code	Value	Description
UNSUPPORTED_CMD	1	
SEQUENCER_NOT_KNOWN	2	
UNAUTHORIZED_ACCESS	3	
LOCKED_MEM_ACCESS	4	
REQUEST_CANCELED	5	
REQUEST_NOT_FOUND	6	
EP_PIN_STUCK	7	error code including EP_Nr and Pin_Nr
EP_NOT_FOUND	8	if a Trigger request refers to a non existing EP
PWM_IN_NO_SIGNAL	9	
REQ_QUEUE_OVFL	10	EP_Nr of Request Queue attached
REQUEST_REJECTED	11	Other than STANDARD request during RC Server initial config phase
CRC_FAILURE	12	CRC of request does not match

Table: Error codes for err-responses

13 ENDPOINTS

13.1 Overview

An endpoint is an entity that has its configuration parameters (like timings, pin configuration, etc.) stored in the EP_config register map of its RC Server. It receives requests from the Client via the RC Server and gives responses as well as notifications via the RC Server back to the client. (Note that the responses might be multicasted.).

Each endpoint listens to one or more configured RC Client(s). Therefore, one or multiple **STREAM_ID**(s) need to be configured. Requests having a not known STREAM_ID are discarded by the RC Server. The **byte_bus_id** is only unique in the context of a STREAM_ID and requests are only passed to the endpoint if both match.

An endpoint may be used or not used in a specific instantiation of a RC PHY. If it is not used (EP_USED bit in EP_config set to 0) then there is no mapping to HW pins and the endpoint is not used to validate any requests.

A used endpoint has control over a physical interface of the RC PHY (except for RC Server endpoint and the WakeUp endpoint).

While the type of the endpoint (e.g. SPI) is fixed by the RC PHY, the instance of the physical interface that is controlled by the endpoint and the associated pins of the RC PHY are configurable within the limits of the RC PHY.

A used endpoint might be enabled or disabled. If enabled, it starts processing requests from its queues. If disabled, the processing is stopped and the RC Client can e.g. preload the queues with a sequence of requests to be executed as soon as the endpoint gets enabled.

The list of parameters for an endpoint has a common part for all types of endpoints. This part includes the type of the endpoint, an endpoint identification (for the purpose of addressing) and a description of the usage (for the purpose of discovery), which could be a system wide unique ID for the function and/or an ID of the device connected to this interface.

If within the RC Server's SERVER_config the ROOT_CLIENT is configured only the ROOT_CLIENT has write access to all RC Server configuration register maps. For other RC Clients than the ROOT_CLIENT write access is only given to EP_FUNC_config of endpoints which are allocated to the RC Client within the EP_ID_config register map.

13.2 Generic part of endpoint register map

The common part of the endpoint register map contains a fixed structure as below per endpoint implemented in an RC PHY. The configuration bit EP_USED hereby indicates if the endpoint in the end is used in the actual instantiation of the RCP_PHY. If used then enabling, request handling, queue size, pin mux and RC Client mapping need to be defined.

Relative address	Parameter	Type		Description
EP0				
0x0000	EP_TYPE	8 bit	R	is an enumerated type giving the index into the TYPE matrix table see below
0x0001	EP_DESC	32 bit	R/W	
0x0004.0	EP_USED	1 bit	R/W	indicating if the RCP_PHY within its actual instantiation makes use of the endpoint or not for EP0 fixed to 1 as EP0 needs to be always implemented
0x0004.1	EP_EN	1 bit	R/W	indicating if the endpoint is enabled to execute requests from its queue; default after power-on is 0: disabled except for EP0 which is always enabled

0x0004.2	EP_CLR_REQ_QUEUE	1 bit	W	writing a 1 clears the EPs request queue, reads always 0
0x0005	EP_REQ_QUEUE_SIZE	16 bit	R/W	Request Queue assigned memory in 32bit words
0x0007.0	EP_REQ_CRC_EN	1 bit	R/W	0 : no CRC checking for this EP requests required 1 : all frames contain an additional CRC EP is in safe mode
0x0007.1	EP_ACK_CRC_EN	1 bit	R/W	0 : no CRC generation 1 : CRC to be generated for acknowledge
0x0007.2	EP_RESP_CRC_EN	1 bit	R/W	0 : no CRC generation 1 : CRC to be generated for responses/notifications
0x0007.3	EP_RESP_TS_EN	1 bit	R/W	0 : no time stamp to be inserted in response 1 : insert time stamp in response
0x0007.4	EP_NOTIFY_TS_EN	1 bit	R/W	0 : no time stamp to be inserted in notification 1 : insert time stamp in notification
0x0007.5	EP_ACK_TS_EN	1 bit	R/W	0 : no time stamp to be inserted in acknowledge 1 : insert time stamp in acknowledge
0x0007.6	EP_RESP_ON_ERROR	1 bit	R/W	0 : no response to be send on error 1 : send a response on error
0x0007.7	EP_NO_RESP_ON_SUCC	1 bit	R/W	0 : send a response on successful execution 1 : no response to be send on successful execution off write request
EP1				
0x0008	EP_TYPE	8 bit	R	is an enumerated type giving the index into the TYPE matrix table see below
...				

Table: EP_GENERIC_config register map

The configuration parameter EP_RESP_ON_ERROR also switches on the gauging of the assigned physical IO pins. If the endpoint tries to set an IO-Pin and the state of the IO-Pin does not follow, then an error will be flagged and the response will be sent. The response shall include the reference to the error causing pin. If the source of an error is a different type (e.g. wrong sequencer addressed) an error response with a corresponding error code is generated.

The EP_TYPE is an enumerated type as follows:

EP_TYPE	Description
0	Server
1	WakeUp Ctrl
2	GPIO
3	SPI
4	I ² C
5	UART
6	LIN

7	PWM_OUT
8	PWM_IN
9	ADC
10	DAC
11	CAN
12	ISELED
13	...

13.3 Request validation

Request for endpoints will be provided via a queue structure. Each endpoint can therefore has its own request queue.

Each endpoint gets its requests via the RC Server based on the **byte_bus_id** within the **stream_id**.

Requests may come within an AVTPDU with a presentation time (i.e. TSCF header). In case the RC PHY does not support time synchronization, the presentation time shall be ignored, and the request(s) executed as if no presentation time were included or dropped depending on the configuration of the RC PHY.

For requests sent in a AVTPDU having a TSCF header with the time stamp marked as invalid (i.e. **tv**=0), the timestamp is ignored. If the reserved bytes in the header are all zero, then the request shall be queued as if the header was in NTSCF format or dropped, depending on configuration. In case the time stamp is uncertain (i.e. **tu**=1), then this shall be executed as if **tu**=0.

13.4 Endpoint status information

For each endpoint status information is readable by the RC Client(s).

The overflow and full bits will be set on the occurring event and need to be actively reset. To do so the RC Client needs to write a zero into the overflow / full bits. The status information is as follows:

Relative address	Parameter	Type	R/W	Description
EP1				
0x0000	EP_STATUS	8 bit	R/W	[0] : EP_IDLE [1]: EP_REQ_EXEC_ONGOING [4] : REQ_QUEUE_OVERFLOW [5]: ACK_QUEUE_FULL [6]: RESP_QUEUE_FULL
0x0001	EP_REQ_QUEUE_STAT	8 bit	R	fill grade of queue in quadlets
EP2				
0x0002	EP_STATUS	8 bit	R/W	[0] : EP_IDLE [1]: EP_REQ_EXEC_ONGOING [4] : REQ_QUEUE_OVERFLOW [5]: ACK_QUEUE_FULL [6]: RESP_QUEUE_FULL
0x0003	EP_QUEUE_STAT	8 bit	R	fill grade of queue in quadlets
EP3				

0x0004	EP_STATUS	8 bit	R/W	[0] : EP_IDLE [1]: EP_REQ_EXEC_ONGOING [4] : REQ_QUEUE_OVERFLOW [5]: ACK_QUEUE_FULL [6]: RESP_QUEUE_FULL
...				

Table: EP_STATUS

1
2

13.5 Endpoint specific usage of evt-bits and byte_msg_payloads

Each endpoint type handles the **byte_msg_payload** in an endpoint type specific way.

While the bit **evt[3]** is used by the RC Client to ask for an acknowledge the other event bits are used to control the usage of the **byte_msg_payload**. For the case of **evt[2]=1** it has been explained that a wait-condition is configured, whereby the exact condition is defined by the **byte_msg_payload** and **evt[1:0]**. A request with **evt[2]=1** does not execute anything, but just returns the status of the interface if **op=0** and when used in a compound operation advances the sequencer state. For the case of **evt[2]=0**, the endpoint will execute on the **byte_msg_payload** and **evt[1:0]**.

The detailed behavior of each endpoint based on the **evt**-bits is described in the following table:

Endpoints	evt[2]	evt[1]	evt[0]	Behavior
RC Server, GPIO	1	0	0	wait for endpoint data to become an exact match to data
			1	wait for endpoint data bits indicated by SET_MSK to be '1'
		1	0	wait for endpoint data bits indicated by CLR_MSK to be '0'
			1	wait for endpoint data bits indicated by MASK to match data from request in this case the byte_msg_payload contains a double data set: MASK and data
	0	0	0	data is used as is
			1	data is interpreted as SET_MSK to the existing data bits set to '1' in the data are set in the endpoint data (OR)
		1	0	data is interpreted as CLR_MSK to the existing data bits set to '1' in the data are cleared in the endpoint data (NAND)
			1	data is interpreted as MASK to update only selected bits by data from request (XOR) Note: an XOR with byte_msg_payload being zero leaves the status unchanged. With op=1, this is a "pure read".
			1	
			1	
SPI	1	0	0	wait for endpoint data to become an exact match to data
			1	wait for endpoint data bits indicated by SET_MSK to be '1'
		1	0	wait for endpoint data bits indicated by CLR_MSK to be '0'
			1	wait for endpoint data bits indicated by MASK to match data from request in this case the byte_msg_payload contains a double data set: MASK and data
	0	0	0	data is used as is and select SPI configuration 0 for data transmission
			1	data is used as is and select SPI configuration 1 for data transmission
		1	0	data is used as is and select SPI configuration 2 for data transmission
			1	data is used as is and select SPI configuration 3 for data transmission
PWM_OUT	1	0	0	wait for act_active time to be below payload second two 8 bits
			1	wait for act_active time to be above payload second two 8 bits
		1	0	wait for act_period time to be below payload first two 8 bits
			1	wait for act_period time to be above payload first two 8 bits
	0	0	0	data is used as is
			1	data is added to the existing data new_active_time = act_active_time + PWM_active from payload new_period = act_period + PWM_period from payload
		1	0	payload data is subtracted from actual used PWM data new_active_time = act_active_time - PWM_active from payload new_period = act_period - PWM_period from payload
			1	data is used as is
PWM_IN	1	0	0	wait for act_active time to be below payload first two 8 bits
			1	wait for act_active time to be above payload first two 8 bits
		1	0	wait for act_period time to be below payload second two 8 bits

	0	0	1	wait for act_period time to be above payload second two 8 bits
			0	for op=0 payload data is ignored; for op=1 request is dropped by RC Server
			1	for op=0 payload data is ignored; for op=1 request is dropped by RC Server
			0	for op=0 payload data is ignored; for op=1 request is dropped by RC Server
ADC	1	0	0	wait for averaged value above the value given by the first two 8 bits
			1	wait for averaged value below the value given by the first two 8 bits
		1	0	wait for averaged value between the value given by the first two 8 bits and the value given by the last two 8 bits
			1	wait for averaged value outside the value given by the first two 8 bits and the value given by the last two 8 bits
	0	0	0	for op=0 payload data is ignored; for op=1 request is dropped by RC Server
			1	for op=0 payload data is ignored; for op=1 request is dropped by RC Server
		1	0	for op=0 payload data is ignored; for op=1 request is dropped by RC Server
			1	for op=0 payload data is ignored; for op=1 request is dropped by RC Server
I ² C, UART, Lin	1	0	0	wait for endpoint data to become an exact match to data
			1	wait for endpoint data bits indicated by SET_MSK to be '1'
		1	0	wait for endpoint data bits indicated by CLR_MSK to be '0'
			1	wait for endpoint data to become an exact match to data
	0	0	0	data is used as is
			1	data is used as is
		1	0	data is used as is
			1	data is used as is

Table: EP specific usage of EVT bits

While doing additions and subtractions neither overflows nor wrap-arounds shall occur. The values are saturated at 0x0000 on the low side and 0xFFFF at the high side.

13.6 End-to-End data protection (safe points)

To enable end-to-end protection of the request each endpoint can be addressed in two modes:

1. Plain command mode
2. Safe command mode which is CRC secured.

The selected mode is defined per endpoint within the EP_GENERIC_config in the RC Server's register map.

The safe command mode behaves like the plain command mode, with the difference that the **byte_msg_payload** is extended by two bytes which shall not be treated as request data but instead contain a 16/32-bit CRC. The CRC is calculated individually for each acf-type (ACF_GBB or ACF_ABB) including the AVTPDU header that contains the acf-type.

That means that if in an AVTPDU multiple acf-types are included, then individual CRCs are calculated for each of the acf-types belonging to an endpoint that is in safe command mode. The seed value of the CRC calculation shall be the endpoint type. This guarantees that data that was meant for e.g. a GPIO endpoint is not used on a SPI endpoint.

For endpoint groups the seed value is generated out of the contained endpoint types within the group.

In case the CRC does not match, the request is dropped.

In responses and acknowledge from an endpoint in safe command mode there shall also be a CRC included. Further details are discussed in the Safety section of this document.

13.6.1 End-to-End protection (EP groups)

For endpoint groups the seed value is generated out of the contained endpoint types within the group. This is done by summing up the individual endpoint types multiplied with their position inside the group and finally adding 256. E.g. a group with the endpoint types [3 2 5] would lead to seed value of $1 \times 3 + 2 \times 2 + 3 \times 5 + 256 = 22 + 256 = 278$.

13.7 Types of endpoints

13.7.1 Endpoint type: RC Server

13.7.1.1 RC Server basic concept

The configuration of the RC Server is stored in the SERVER_config register map, as described in RC Server Register Map. Therefore, the RC Server as endpoint is not included in the EP_FUNC_config register maps.

The RC Server's register map can be accessed for read and write by the RC Client via ACF_ABB with the servers configured **byte_bus_id** (which is by default zero).

The server shall support sending a cyclic heartbeat. This can be achieved by configuring a response queue to send with a fixed periodicity. If the queue would be empty an empty PDU (just NTSF header, but no **acf_types**) will be sent.

Alternatively, a trigger request -if supported- can be queued that cyclically generates a notification with the read result from a status or diagnostic register.

13.7.1.2 Server EP register access

The Server can be addressed via an ACF_ABB in the same way as any other functional endpoint. But while for other endpoints the payload carries a request in an endpoint specific way, the server handles the payload as a read or write access to its register map.

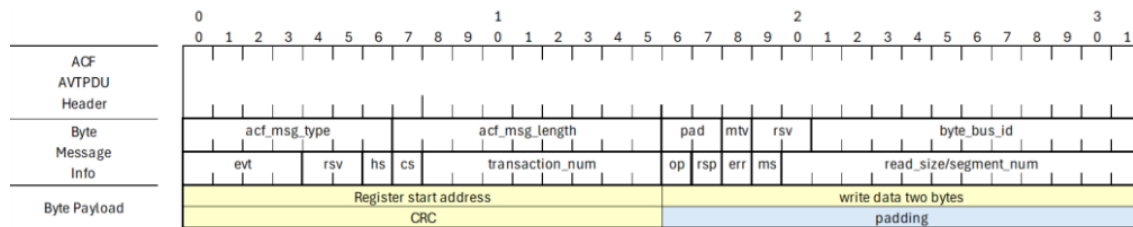


Figure 17 - ACF_ABB for RC Server register write access in safe mode

The payload consists of a 16-bit address, a CRC if safe mode is requested and data to be written to the address. Effective number of bytes to be written = $(acf_msg_length - 3) \times 4 - pad$.

The handling of the data depends on **evt[2:0]** so that the bits are either SET, OR'ed, AND'ed or XOR'ed to the target registers content.

A request with **byte_msg_payload** "0x0000 0000" and **evt**=x001 (OR) results in "no effect". Writing data to read only registers has no effect and request is confirmed normally. Writing to a write prohibited register (e.g. lock bit for map set) creates a response with **err**=1 and an error code UNAUTHORIZED_ACCESS.

Each of the defined _config register maps can be accessed via its base address and adding the relative address from _config map description. For the EP_FUNC_config the sequence of the functional descriptions of the endpoints is determined by their sequence in the EP_GENERIC_config.

Example: For a system with 3 endpoints (GPIO, PWM_OUT, ADC) the full EP_FUNC_config including the wake-up and sleep endpoint would look like this:

relative address	Parameter	Type	R/W	Description
WakeUp				
0x0000	EP_LEN	8 bit	R	length of this description [0x08 + Nr_of_WuP_IOs]
0x0001	WuP_BASE_CLK	16 bit	R	WuP base clock in kHz
0x0003	WuP_CLK_DIVIDER	8 bit	R/W	
0x0004	WuP message repeat	16 bit	R/W	delay between two WuP messages in clock ticks

0x0006	Nr_of_WuP_IOs	8 bit	R/W	Nr of IO pins to gauge for activity
0x0007	IO_SRC_1	16 bit	R/W	[10: 0] IO pin number [15:11] IO behavior
0x0009	IO_SRC_2	16 bit	R/W	[10: 0] IO pin number [15:11] IO behavior
0x000B	...		R/W	
0x000B	WuP DEBOUNCE	8 bit	R/W	time a signal change needs to be stable to trigger wake up (in clock tics)
GPIO				
0x000C	EP_LEN	8 bit	R	length of this description [0x05]
0x000D	GPIO_BASE_CLK	16 bit	R	GPIO system clock
0x000F	GPIO_MAX_IO_NR	8 bit	R	defines the maximum IO Pin number supported by the GPIO EP
0x0010	GPIO_CLK_DIVIDER	8 bit	R/W	divider for the GPIO base clock to generate the GPIO_CLK
0x0011	GPIO_IN_DEBOUNCE	8 bit	R/W	debounce time in GPIO_CLK cycles
PWM_OUT				
0x0012	EP_LEN	8 bit	R	length of this description [0x06]
0x0013	PWM_BASE_CLK	16 bit	R	PWM system clock
0x0015	PWM_CLK_DIVIDER	8 bit	R/W	divider for the PWM base clock to generate the PWM_CLK
0x0016.0	PWM_INV_POL	1 bit	R/W	invert polarity of PWM output 0: active phase is high 1: active phase is low
0x0016.1	PWM_PO_IDLE_STATE	1 bit	R/W	Idle state of primary PWM output 0: LOW 1: HIGH
0x0016.1	PWM_SO_IDLE_STATE	1 bit	R/W	Idle state of inverted PWM output 0: LOW 1: HIGH
0x0017	PWM_RE_DEL	8 bit	R/W	delay in clock ticks for rising edge on PWM_out and PWM_outn
ADC				
0x0018	EP_LEN	8 bit	R	length of this description [0x0D]
0x0019	ADC_BASE_CLK	16 bit	R	ADC system clock
0x001B	ADC_CLK_DIVIDER	8 bit	R/W	generate ADC_CLK time reference
0x001C	SAMPLE_INTERVAL	8 bit	R/W	nr of ADC_CLK cycles between two sample
0x001D	SAMPLES_PER_MEAS	8 bit	R/W	Samples per measurement cycle
0x001E	AVG_INTERVAL	8 bit	R/W	Nr of measurements to be averaged

0x001F	COMBINE_NR_VAL	8 bit	R/W	Nr of output values to be combined in one notification or response
0x0020	ADC_NR_OF_BITS	8 bit	R/W	defines the number of bits to be used of the ADC reading value
0x0021	ADC_MIN_TRIGGER	16 bit	R/W	ADC trigger signal to be set low below this threshold
0x0023	ADC_MAX_TRIGGER	16 bit	R/W	ADC trigger signal to be set high above this threshold

1
2

13.7.2 Endpoint type: Wakeup control

13.7.2.1 Wakeup basic concept

The WakeUp endpoint is a special endpoint and as this fixed to the endpoint nr 1, as it is the only endpoint which stays active in Sleep mode of the RC PHY.

When the RC PHY is in sleep mode the endpoint listens to configurable IO signals for activity as well as the network interface as wake-up source. Which and how many inputs can be observed is a device implementation specific feature and might therefore be limited in the selection of the IOs.

The configuration of the IO pins to listen to is done via the endpoint's configuration. When the WakeUp endpoint detects activity during sleep mode it wakes up the RC PHY and specially the network interface. After establishing a network connection, the WakeUp endpoint sends repetitive messages on the wake-up source to the RC Client which initiated the sleep mode. The timing interval is configurable.

Repetition of the notification continues until the RC Client acknowledges the wake-up to the WakeUp endpoint, by sending a blank request to the WakeUp endpoint.

13.7.2.2 Wakeup functional configuration

The **byte_bus_id** to access the WakeUp EP is also defined via the EP_ID_map.

The WakeUp EP needs a mapping of the IO-Pins it listens to wake up.

A debounce time for the signals is specified to suppress wake-up on noise.

Relative address	Parameter	Type	R/W	Description
WakeUp				
0x0000	EP_LEN	8 bit	R	length of this description [0x08 + Nr_of_WuP_IOs]
0x0001	WuP_BASE_CLK	16 bit	R	WuP base clock in kHz
0x0003	WuP_CLK_DIVIDER	8 bit	R/W	
0x0004	WuP message repeat	16 bit	R/W	delay between two WuP messages in clock tics
0x0006	Nr_of_WuP_IOs	8 bit	R/W	Nr of IO pins to gauge for activity
0x0007	IO_SRC_1	16 bit	R/W	[10: 0] IO pin number [15:11] IO behavior
0x0008	IO_SRC_2	16 bit	R/W	[10: 0] IO pin number [15:11] IO behavior
0x0009	...		R/W	
0x0009	WuP DEBOUNCE	8 bit	R/W	time a signal change needs to be stable to trigger wake up (in clock tics)

Table: EP1 WakeUp/Sleep functional description

IO_SRC[15:11]	Description
00000	Pin inactive
00001	rising edge
00010	falling edge
00011	both edges
00100	high level
00101	low level
all other combinations	reserved

Table: WakeUp Source Pin Configuration

13.7.2.3 Wakeup request and notification handling

To bring the RCP_PHY into sleep mode the RC Client has to send a specific request to the WakeUp endpoint.

Based on the sleep request the RC Server will:

1. stop entering incoming requests into endpoint queues.
2. EPs to signal that there are no more pending execution time base requests nor standard requests in the queue
3. as soon as all EPs are idle and the responder queues are empty (all responses sent):
13. send an acknowledge to sleep request
14. bring the RCP_PHY to sleep mode.

The WakeUp endpoint sleep request is sent as:

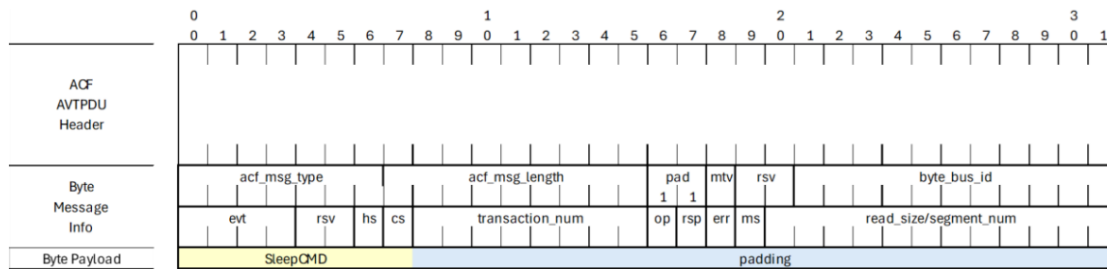


Figure 18 - Endpoint sleep request

SleepCMD is coded as 0xA5.

13.7.3 Endpoint type: SPI (SPI controller)

13.7.3.1 SPI EP basic concept

This SPI endpoint is an SPI controller.

An “SPI responder” would be a different type of endpoint, which is currently not foreseen.

An SPI Endpoint supports two modes of connecting to SPI devices, a point2point connection via a single "chip select" (SCSN) and a parallel SPI implementation where the clock and data lines are attached to a multitude of SPI devices which are having each its own SCSN.

As in parallel SPI mode the bus attached devices not necessarily have all the same SPI communication configuration the SPI endpoint supports up to 4 pre-configurable sets of SPI configurations, which are selected in the SPI request via the event-bits **evt[1:0]** with **evt[2]** being set to '0'.

The four configurations allow the physical interface also to have up to 4 dedicated SCSN lines served directly by the physical interface. If the physical interface does not support multiple SCSNs or if more than 4 devices are attached to the parallel SPI bus additional SCSN signals can be generated by using additional GPIO pins.

The list of configurable parameters of an SPI endpoint is given in the next section. The Client can request the SPI endpoint to send one or more bytes that are delivered as **byte_msg_payload** of an ACF_ABB/GBB to the endpoint. This is described in detail in Annex O in IEEE1722b.

The execution of this request might be “immediate”(asap), “timed” or “conditional”. Requests are accepted in ACF_ABB and ACF_GBB. Optionally the Client can request the SPI endpoint to wait by **evt[2]=1** until a pre-defined bit sequence has been received from the connected device. Since evt[1:0] are used to define whether a mask is used, the byte_msg_payload in the wait-request is compared against the data received on SPI0. Waiting for specific data on the other SPI channels of the same EP (if those are supported) is excluded.

An SPI endpoint creates the following trigger events depending on physical interfaces implementation:

- 0: SPI execution done
- 1: reserved
- 2: CSN0 asserted
- 3: CSN0 de-asserted
- 4: CSN1 asserted
- 5: CSN1 de-asserted
- 6: CSN2 asserted
- 7: CSN2 de-asserted
- 8: CSN3 asserted
- 9: CSN3 de-asserted

13.7.3.2 SPI EP functional configuration

The generic part of the register map that is common for all types of endpoints is described in section Generic part of endpoint register map.

The functional configuration parameters listed in this section are SPI function dedicated. Access to these registers is handled via the RC Server EP_functional_config register_map. The access to the functional parameters can be locked in the generic endpoint configuration.

Relative Address	Parameter	Type	R/W	Description
SPI				
0x0000	EP_LEN	8 bit	R	length of this description [0x04+ SPI_Nr_Configs * 0x06]
0x0001	SPI_BASE_CLK	16 bit	R	SPI base clock in kHz
0x0002	SPI Nr of Configs	8 bit	R/W	how many SPI configurations are configured
SPI_CFG1				
0x0003	SPI_CLK_DIVIDER	8 bit	R/W	divider for the SPI base clock to generate the SPI_SCLK

0x0004.0	SCLK_POL	1 bit	R/W	polarity of SCLK signal 0: normal 1: inverted
0x0004.1	SCLK_PH	1 bit	R/W	sample phase of SCLK 0: sample on rising edge 1: sample on falling edge
0x0004.2	SCSN_POL	1 bit	R/W	polarity of CS signal 0: CS is low active 1: CS is high active
0x0004.3	USE_HW_SCSN	1 bit	R/W	SPI HW implementation has multiple CS 0: common CS is used 1: HW_SCSN0 is used
0x0005	SCSN_SCLK_DEL	8 bit	R/W	delay between assertion of SCSN to first SCLK edge in SPI_BASE_CLK tics
0x0006	SCLK_SCSN_DEL	8 bit	R/W	delay for deassertion of SCSN after last SCLK edge in SPI_BASE_CLK tics
0x0007	SPI_MAX_BITS	8 bit	R/W	maximum number of consecutive bits send on bus without pause
0x0008	SPI_TRAIL_TIME	8 bit	R/W	minimum time between two SPI transfers in SPI_SCLK cycle
SPI_CFG2				
0x0009	SPI_CLK_DIVIDER	8 bit	R/W	divider for the SPI base clock to generate the SPI_SCLK
0x000A.0	SCLK_POL	1 bit	R/W	polarity of SCLK signal 0: normal 1: inverted
0x000A.1	SCLK_PH	1 bit	R/W	sample phase of SCLK 0: sample on rising edge 1: sample on falling edge
0x000A.2	SCSN_POL	1 bit	R/W	polarity of CS signal 0: CS is low active 1: CS is high active
0x000A.3	USE_HW_SCSN	1 bit	R/W	SPI HW implementation has multiple CS 0: common CS is used 1: HW_SCSN1 is used
0x000B	SCSN_SCLK_DEL	8 bit	R/W	delay between assertion of SCSN to first SCLK edge in SPI_BASE_CLK tics
0x000C	SCLK_SCSN_DEL	8 bit	R/W	delay for deassertion of SCSN after last SCLK edge in SPI_BASE_CLK tics
0x000D	SPI_MAX_BITS	8 bit	R/W	maximum number of consecutive bits send on bus without pause
0x000E	SPI_TRAIL_TIME	8 bit	R/W	minimum time between two SPI transfers in SPI_SCLK cycle
SPI_CFG3				
...				

Table: SPI functional configuration in EP_FUNC_config

Additionally, the pin mapping of the endpoint is handled within the RC Server HW_config register map, binding SCLK, SCSN, POCL, and PICO.

If a parallel SPI structure shall be implemented depending on the SPI HW interfaces implementation up to 4 SCSN signals may be handled by the SPI endpoint via its configurations. If the interface implementation just supports one SCSN it shall not be mapped to an IO-Pin. Instead, the SCSN of the individual devices on the parallel SPI bus shall be handled via a GPIO endpoint. Easy synchronization of the GPIO asserting and de-asserting can be achieved by the GPIO being controlled via a trigger on the SPI SCSN signal.

To allow different bitrates and polarities for the different devices attached to a parallel SPI bus the SPI bus functional configuration supports 4 different configurations in parallel. Which configuration is used within an SPI request is determined by **evt[1:0]**.

13.7.3.3 SPI EP request handling

SPI request data, except **evt** bits are well defined in IEEE1722. See Annex O in IEEE1722: _IEEE Draft Std 1722-2016-Rev-5th WG_Proposed_Comment_Dispatch-v01.pdf.

The interpretation of the **evt-bits** is described in the section "The usage of evt bits in requests".

As the SPI is able to wait for configurable input data and then trigger the start of a compound operation it needs to keep the last received data internally available for checking (wait-operation) up to the length of the readsize defined with the acf-type.

If the SPI stops execution (see **cs/hs** bits) – then SPI endpoint is in error state (->debug register). And the SPI endpoints enable bit in the EP_config is reset. The error status needs to be cleared (by the Client) and the endpoint needs to be enabled again to continue operation.

The Client can also decide to clear one or more queues of this endpoint before continuing operation. For safety reasons it might be important to read the state of SPI pins (check against stuck at VDD/GND, latching etc. errors). A clamped pin shall raise a flag in the server's diagnostic registers and set the "error" flag in all responses of this endpoint.

13.7.4 Endpoint type: GPIO

13.7.4.1 GPIO EP basic concept

A GPIO endpoint maps bits from a **byte_msg_payload** to physical pins.

Each GPIO endpoint can handle up to 32 IOs.

By this a GPIO endpoint provides access to digital input and outputs pins.

A GPIO endpoint serves multiple IO signals. For each signal a pin of the RCP Phy implementation can be selected. And each IO signal is mapped to a bit of the **byte_msg_payload**. See drawing below. A GPIO endpoint has a configurable “debounce time” that is valid for all its inputs.

For each pin the direction (input/output) can be configured. For each pin configured as output the drive strength can be configured.

For each pin configured as output it can be configured, if this is push-pull, open-drain or open source.

Each GPIO pin can be assigned to a bit position of the **byte_msg_payload**. The first bit of the **byte_msg_payload** after the **readsize/segment_num** field is “bit 0” the last bit of that quadlet is “bit 31”.

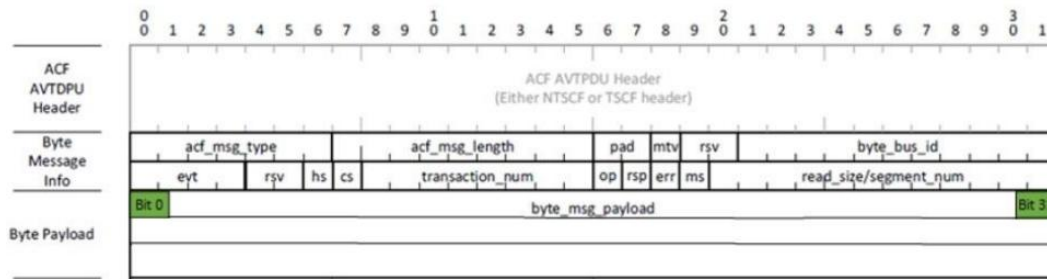


Figure 19 - GPIO frame

A GPIO endpoint shall be able to wait for a configurable state and then trigger the start of a compound operation and/or send a response.

When a “wait” is in the conditional queue, then a response is sent only if the configured wait condition occurred. Output pins can be set to a defined state and the current state can be changed by a logical operation (NAND, OR, XOR). Generating a pulse for a defined time is NOT a function of the GPIO endpoint. This either needs to be managed by the Client (i.e. sending two commands; may be with presentation times) by a compound or a trigger operation. The Client can set the outputs and read the inputs.

The GPIO Endpoint can also be triggered by another endpoint to generate a notification.

Requests are accepted in ACF_ABB and ACF_GBB.

The GPIO endpoint generates triggers for all its mapped IO-pins on the change, falling and rising edge.

The reference numbers of the triggers are build such that change, rising and falling edges of a GPIO are always in sequence:

- 0: GPIO EP request execution done
- 1: GPIO[0] signal change
- 2: GPIO[0] rising edge
- 3: GPIO[0] falling edge
- 4: GPIO[1] signal change
- 5: GPIO[1] rising edge
- 6: GPIO[1] falling edge
- 7: GPIO[2] signal change
- 8: GPIO[2] rising edge
- ...
- 96: GPIO[31] falling edge

13.7.4.2 GPIO EP functional configuration

Relative Address	Parameter	Type	R/W	Description
GPIO				
0x0000	EP_LEN	8 bit	R	length of this description [0x05]
0x0001	GPIO_BASE_CLK	16 bit	R	GPIO system clock
0x0003	GPIO_MAX_IO_NR	8 bit	R	defines the maximum IO Pin number supported by the GPIO EP
0x0004	GPIO_CLK_DIVIDER	8 bit	R/W	divider for the GPIO base clock to generate the GPIO_CLK
0x0005	GPIO_IN_DEBOUNCE	8 bit	R/W	debounce time in GPIO_CLK cycles

Table: GPIO functional configuration in EP_FUNC_config

13.7.4.3 GPIO EP request handling

All defined request formats are valid for GPIO EPs.

The **evt** bits are assessed as described in "The usage of evt bits".

GPIO endpoints update their internal read register and deliver responses (current IO status) only upon a request with op set to 1.

A WAIT request assesses the internal GPIO read register versus the WAIT condition in the **byte_msg_payload** of the request. The trigger events a GPIO endpoint generates are based on the physical state of to IO Pins.

A request with data "0x0000 0000" and **evt**=0x001 (SETMSK) results in a "no effect".

The server collects the delivered responses and notifications in the configured transmit queues and sends them according to the configuration of the responder queues.

For safety reasons it the state of the output pin can be checked vs the setting from the GPIO endpoint. This is done if in the EP_config the EP_NOTIFICATION_ON_Error is set and also a notification indicating the error and the error causing pin shall be sent.

13.7.5 Endpoint type: PWM_OUT

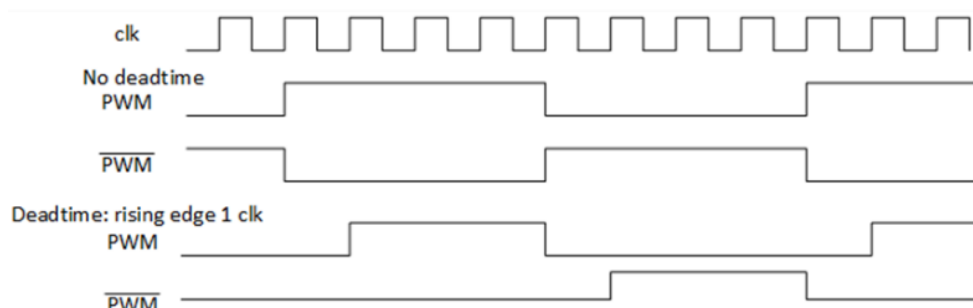
13.7.5.1 PWM_OUT EP basic concept

Each PWM_OUT endpoint provides one pair of complementary PWM signals. The use of the inverted PWM signal is optional.

For each PWM_OUT endpoint at least a first output pin needs to be configured in the HW_config.

For each PWM_OUT endpoint a second output pin can be configured which has the opposite phase of the first output pin.

When such inverted output pin is configured the delay of the rising edges of the primary and inverted output can be defined in clock cycles of the PWM source clock. This is to support “break before make” behavior in half- and full bridge drivers. For trigger signal generation the delayed signal is used.



A PWM endpoint creates two trigger events:

0: PWM request exec done

1: at the start of the cycle

2: in the middle of the active pulse.

Event 1 can be used by other PWM_OUT endpoints to create signals that are phase correlated.

Event 2 can e.g. be used to synchronize an ADC endpoints to sample a voltage in the active phase of the PWM_OUT endpoint signal.

13.7.5.2 PWM_OUT EP functional configuration

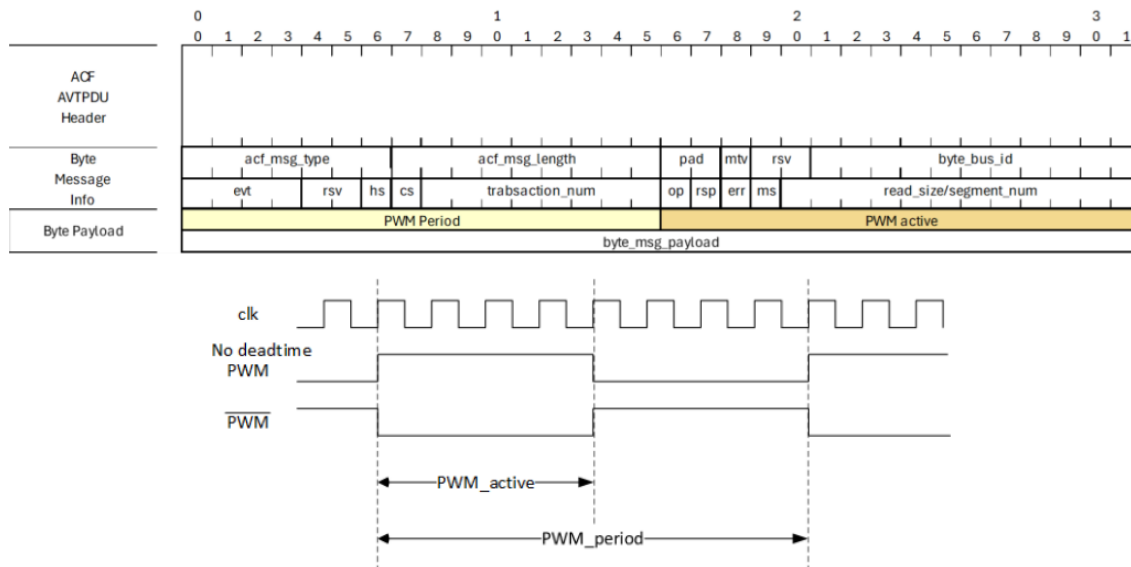
The functional configuration of the PWM_OUT endpoint configures mainly the clock used for PWM generation, as well as the polarity of the PWM output signals and a potential delay for the first edge (start of active phase).

Relative Address	Parameter	Type	R/W	Description
PWM_OUT				
0x0000	EP_LEN	8 bit	R	length of this description [0x06]
0x0001	PWM_BASE_CLK	16 bit	R	PWM system clock
0x0003	PWM_CLK_DIVIDER	8 bit	R/W	divider for the PWM base clock to generate the PWM_CLK
0x0004.0	PWM_INV_POL	1 bit	R/W	invert polarity of PWM output 0: active phase is high 1: active phase is low
0x0004.1	PWM_PO_IDLE_STATE	1 bit	R/W	Idle state of primary PWM output 0: LOW 1: HIGH
0x0004.1	PWM_SO_IDLE_STATE	1 bit	R/W	Idle state of inverted PWM output 0: LOW 1: HIGH

0x0005	PWM_RE_DEL	8 bit	R/W	delay in clock ticks for rising edge on PWM_out and PWM_outn
--------	------------	-------	-----	--

13.7.5.3 PWM_OUT EP request handling

The PWM request, response and notification contain two values in the payload. The active time of the PWM signal as well as the period of the signal in clock ticks of the clock selected for this PWM endpoint.



With the request the PWM endpoint is started and generates continuously a PWM signal on its assigned output pins until stopped. To stop a running PWM generation a request with PWM_Period of '0' shall be issued.

In case the request is a trigger configuration, the first two bytes of the **byte_msg_payload** state the phase shift (and not the period). With this a PWM output can be synchronized with a fixed phase relation to another PWM_OUT endpoint.

During PWM generation the output pin will also be used as input and checked that they really toggle. If not, an error shall be signaled.

13.7.6 Endpoint type: PWM_IN

13.7.6.1 PWM_IN EP basic concept

A PWM_IN endpoint basically measure an incoming PWM signal and determines its active signal time as well as the period in clock cycles of the configured PWM_IN_CLK.

If the period of the incoming signal exceeds the configured maximum allowed period or if the external PWM source stops, the PWM_IN endpoint will detect this and if enabled within the EP_config (EP_RESP_ON_ERR) notify the RC Client with an error notification message.

The PWM_IN endpoint generates two trigger events:

0: rising edge of PWM_IN signal

1: falling edge of PWM_IN signal

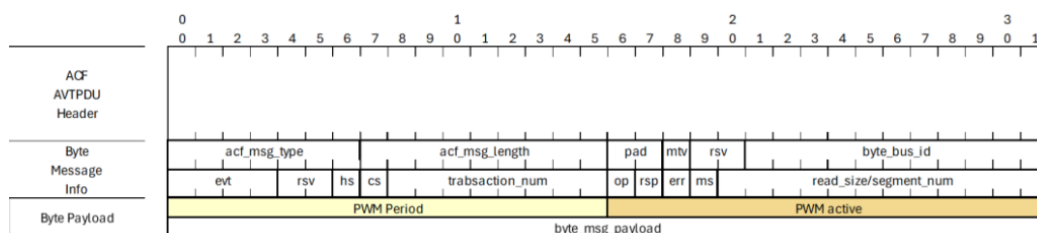
13.7.6.2 PWM_IN EP functional configuration

Relative Address	Parameter	Type	R/W	Description
PWM_IN				
0x0000	EP_LEN	8 bit	R	length of this description [0x06]
0x0001	PWM_BASE_CLK	16 bit	R	PWM system clock
0x0003	PWM_CLK_DIVIDER	8 bit	R/W	divder for the PWM base clock to generate the PWM_CLK
0x0004.0	PWM_POL	1 bit	R/W	polarity of PWM input 0: high during active phase 1: low during active phase
0x0004.1	PWM_ERR_ON_MP	1 bit	R/W	0: if MAX PERIOD is exceeded, invalidate measurement and wait for new active phase of signal 1: if MAX_PERIOD is exceeded stop measurement and signal error if error notification is enabled in EP_config
0x0005	PWM_MAX_PERIOD	16 bit	R/W	MAX PERIOD in PWM_CLK cycle - if exceeded measurement is invalid - err response

13.7.6.3 PWM_IN EP request handling

The PWM_IN request, response and notification contain two values in the payload. The values in a request are ignored in case **evt[2]=0**.

Responses and notifications include the measured active time of the PWM signal as well as the period of the signal in clock ticks of the clock selected for this PWM_IN endpoint.



- 1 The interpretation of the **evt**-bits is described in the section "The usage of **evt** bits in requests".
- 2

13.7.7 Endpoint type: I²C (I²C Controller)

13.7.7.1 I²C EP basic concept

The I2C endpoint is an I²C controller. It drives a single I²C bus.

The Client can request the I²C endpoint to send one or more bytes that are delivered as **byte_msg_payload** of an ACF_ABB to the endpoint. This is described in detail in Annex M in IEEE1722b. Requests are accepted in ACF_ABB and ACF_GBB.

Optionally the Client can request the I²C endpoint to wait for a pre-defined bit sequence has been received from a connected device.

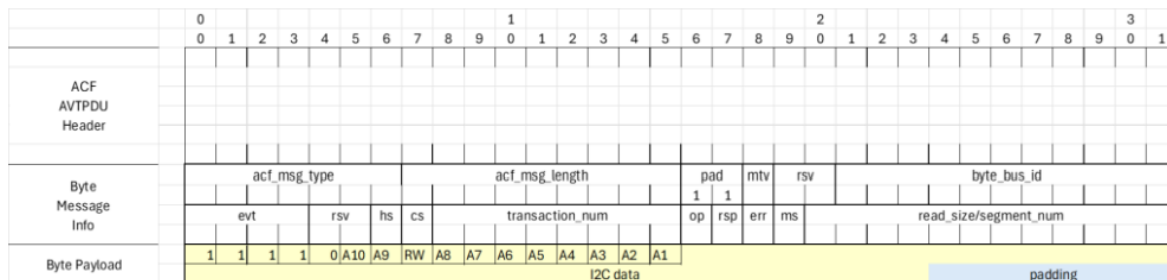
13.7.7.2 I²C EP functional configuration

Relative Address	Parameter	Type	R/W	Description
I ² C				
0x0000	EP_LEN	8 bit	R	length of this description [0x06]
0x0001	I2C_BASE_CLK	16 bit	R	I ² C system clock
0x0003	I2C_CLK_DIVIDER	8 bit	R/W	divder for the I ² C base clock to generate the I2C_CLK
0x0004	I2C_MODE	8 bit	R/W	selects between: 0: Standard Mode 100kbit/s 1: Fast Mode 400kbit/s 2: Fast Mode plus 1Mbit/s 3: High-speed mode 1.7Mbit/s 3: High-speed mode 3.4Mbit/s 4: Ultra-fast mode 5 Mbit/s
0x0005	I2C_TRAIL	8 bit	R/W	Minimum trail time between two I ² C transactions in I2C clock cycle

13.7.7.3 I²C EP request handling

The byte msg payload is the I2C payload including the address. The I2C endpoint does not know whether there is a 7- or 10-bit address, since the endpoint is just transparent.

The example below shows a payload for an I2C transfer with 10bit address and 5 bytes data.



13.7.8 Endpoint type: UART

13.7.8.1 UART EP basic concept

The UART interface allows transmission and reception of data. These two processes are independent from each other.

A request from a Client with **evt[2]=0** leads to a transmission of data to an external connected device. A request from a Client with **evt[2]=1** leads to a response to the RC Client, when the expected data has been received.

Data received from the external connected device will be send in notifications to the Client, if a notification queue is configured for the UART endpoint.

13.7.8.2 UART EP functional configuration

Relative Address	Parameter	Type	R/W	Description
UART				
0x0000	EP_LEN	8 bit	R	length of this description [0x0B]
0x0001	UART_BASE_CLK	16 bit	R	UART system clock
0x0003	UART_CLK_DIVIDER	8 bit	R/W	divider for the UART base clock to generate the UART_CLK
0x0004	UART_BIT_PERIOD	16 bit	R/W	bit period in UART_CLK cycles
0x0006	UART_NR_BITS	8 bit	R/W	Nr of data bits
0x0007.0	UART_PARITY_EN	1 bit	R/W	0: no parity 1: parity according to PARITY_POL
0x0007.1	UART_PARITY_POL	1 bit	R/W	0: odd parity 1: even parity
0x0007.2	UART_RTS_EN	1 bit	R/W	0: no RTS before data transmission 1: RTS will be generated before data transmission
0x0007.3	UART_CTS_EN	1 bit	R/W	0: after RTS set transmission starts two waiting for CTS 1: after CTS data transmission starts
0x0008	UART_STOP_BITS	8 bit	R/W	Number of stop bits in half stop bits
0x0009	UART_TIMEOUT	8 bit	R/W	receiver timeout in UART_CLK cycles
0x000A	UART_TRAIL	8 bit	R/W	trail time between two transmissions in UART_CLK cycles

to be discussed: Adding "Expected number of bytes per reception interval" and "idle time out for reception"

13.7.8.3 UART EP request handling

The **byte_msg_payload** in the request is the UART payload (plus padding if needed and plus CRC if applicable).

The example below shows a payload of 19 bytes.

- 1
- 2

13.7.9 Endpoint type: ADC

13.7.9.1 ADC basic concept

This endpoint type is for ADC's with a resolution of up to 16bits. ADCs with a higher resolution than 16 bit are a different type of endpoint.

One ADC endpoint serves one ADC channel.

For each ADC EP an analog input pin on the RCP Phy package needs to be selected.

Reading ADC values from an endpoint can be e.g. triggered by a PWM output trigger event by using a Trigger request, which triggers on the PWM EP.

The ADC endpoint can alternatively be configured to generate responses cyclically with a configurable cycle time (by using a trigger request which triggers on the ADC EP itself having finished a measurement).

Anyhow, the RC Client may request ADCs reading at any time.

Once a reading is requested, then the ADC samples. If at least one of "SAMPLES_PER_MEAS" or "AVG_INTERVAL" is larger than 1, then further samples are taken with a timely distance of one SAMPLE_INTERVAL (given in multiples of the sample clk). For the readings acquired during the averaging interval (given in number of samples (an implementation may decide to fix this to 1) the average is calculated. Finally, the COMBINE_NR_VAL defines how many measurement values (e.g. averaging results) will be put in one response frame.

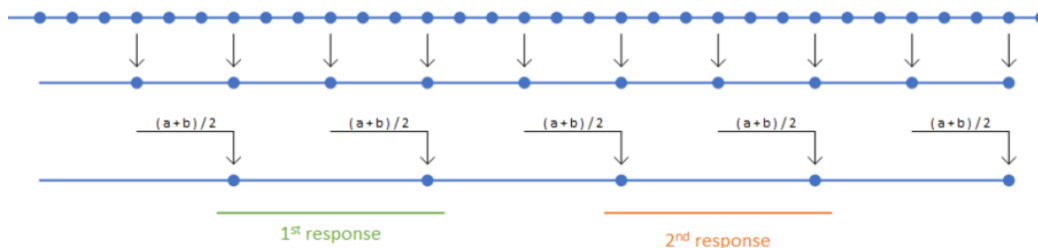


Figure 20 - Example response generation with given parameters

An ADC endpoint can generate five trigger events:

- 0: ADC output value falls below ADC_MIN_TRIGGER
- 1: ADC output value rises above ADC_MIN_TRIGGER
- 2: ADC output value falls below ADC_MAX_TRIGGER
- 3: ADC output value rises above ADC_MAX_TRIGGER
- 4: ADC finished measurement cycle

13.7.9.2 ADC EP functional configuration

The functional configuration of the ADC endpoint mainly configures the sampling interval and handling of samples.

Relative Address	Parameter	Type	R/W	Description
ADC				
0x0000	EP_LEN	8 bit	R	length of this description [0x0D]
0x0001	ADC_BASE_CLK	16 bit	R	ADC system clock
0x0003	ADC_CLK_DIVIDER	8 bit	R/W	generate ADC_CLK time reference
0x0004	SAMPLE_INTERVAL	8 bit	R/W	nr of ADC_CLK cycles between two sample
0x0005	SAMPLES_PER_MEAS	8 bit	R/W	Samples per measurement cycle

0x0006	AVG_INTERVAL	8 bit	R/W	Nr of samples to be averaged
0x0007	COMBINE_NR_VAL	8 bit	R/W	Nr of output values to be combined in one notification or response
0x0008	ADC_NR_OF_BITS	8 bit	R/W	defines the number of bits to be used of the ADC reading value (<=16)
0x0009	ADC_MIN_TRIGGER	16 bit	R/W	ADC trigger signal to be set low below this threshold
0x000B	ADC_MAX_TRIGGER	16 bit	R/W	ADC trigger signal to be set high above this threshold

13.7.9.3 ADC request handling

The ADC request has no payload in case of **evt[2]=0**. Responses contain as much measurement values as requested by half the read size of a request.

A measured value may contain the average of multiple ADC samples depending on the ADC EP functional configuration.

Example response with a single measurement value.

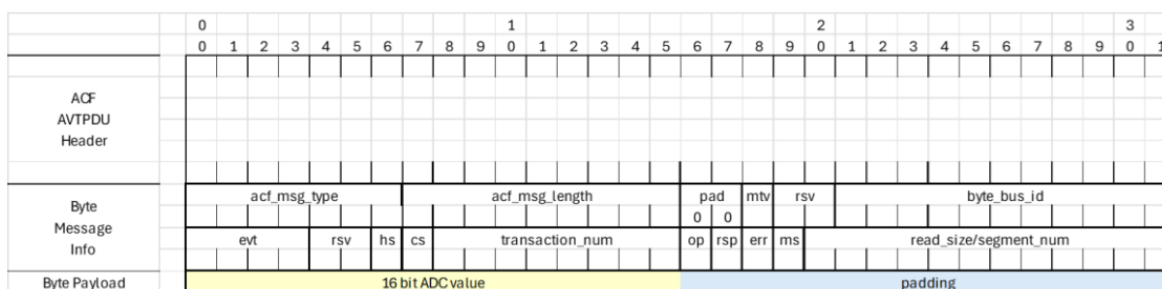


Figure 21 - Response with a single measurement value

The interpretation of the **evt-bits** is described in the section "The usage of evt bits in requests".

Examples:

1) ADC sampling of 8 measurements: SAMPLE_INTERVAL is set to 1us AVERAGE_INTERVAL is set to 1, so no averaging (0 does the same)

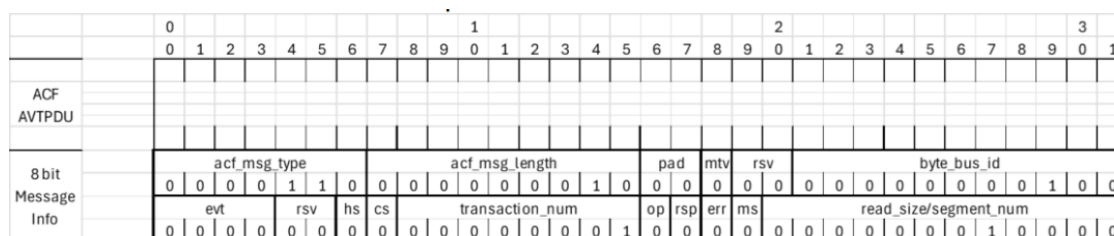


Figure 22 – RC Client sends a standard read request

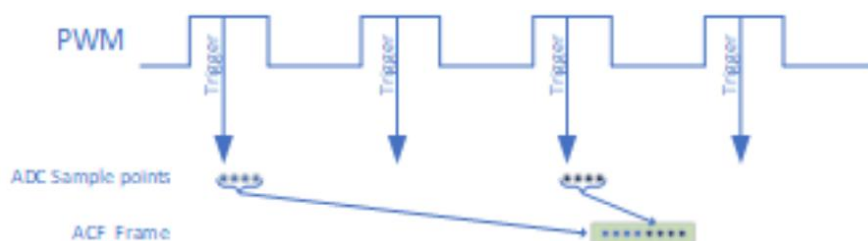
The readsize is set to 16 such that 8 measurements are expected in return.

The ADC endpoint will now do 8 measurements with a temporal distance of 1us and combine them into a response:

		0															1															2															3														
		0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1																												
ACF AVTPDU																																																													
8 bit Message Info		acf_msg_type							acf_msg_length							pad		mtv		rsv		byte_bus_id																																							
		0	0	0	0	1	1	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0																												
		evt			rsv		hs	cs	transaction_num							op	rsp	err	ms	read_size/segment_num																																									
		0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	0										0	1	0	0	0	0																										
		16 bit ADCvalue 0															16 bit ADCvalue 1																																												
		16 bit ADCvalue 2															16 bit ADCvalue 3																																												
		16 bit ADCvalue 4															16 bit ADCvalue 5																																												
		16 bit ADCvalue 6															16 bit ADCvalue 7																																												

Figure 23 - ADC response with 8 measurements

2) ADC configured to trigger on PWM to sample every second PWM cycle 4 measurements and combine 8 measurements per acf frame



PWM EP is EP_Nr: 11 and Trigger_Nr of the PWM EP is 1: mid of active cycle

SAMPLE INTERVAL is set to 1us

AVERAGE_INTERVAL is set to 1, so no averaging (0 does the same)

COMBINE_NR_VAL is set to 8

		0										1										2										3									
		0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1								
ACF	AVTPDU																																								
8 bit Message Info		acf_msg_type								acf_msg_length								pad		mtv		rsv		byte_bus_id																	
		0	0	0	1	1	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0									
		1	1	1	1	1	1	0	0	0	0	0	1	0	1	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	1								
		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1									
		evt				rsv		hs	cs	transaction_num								op		rsp	err	ms	read_size/segment_num																		
		0	0	0	0			0			0	0	0	0	0	0	1	0	0	0	0					0	0	0	0	0	0	0	0								

Figure 24 - RC Client sends a Trigger request

With every second trigger of the PWM the ADC will now generate 4 measurements with a time distance of 1 μ s.

After two trigger request executions the ADC endpoint has gathered 8 values and generates an acf frame:

		0								1								2								3							
		0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1
ACF AVTPDU																																	
8 bit Message Info		acf_msg_type								acf_msg_length								pad		mtv		rsv		byte_bus_id									
		0	0	0	1	1	1	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
		evt		rsv		hs	cs	transaction_num								op	rsp	err	ms	read_size/segment_num													
		0	0	0	1													0	1	0	0				0	0	0	0	0	0	0	0	0
		16 bit ADC value 0 1st PWM cycle																16 bit ADC value 1 1st PWM cycle															
		16 bit ADC value 2 1st PWM cycle																16 bit ADC value 3 1st PWM cycle															
		16 bit ADC value 0 3rd PWM cycle																16 bit ADC value 1 3rd PWM cycle															
		16 bit ADC value 2 3rd PWM cycle																16 bit ADC value 3 3rd PWM cycle															

13.7.10 Endpoint type: LIN commander

13.7.10.1 LIN EP basic concept

The LIN EP is a LIN master. It sends the bytes provided by the RC Client in the **byte_msg_payload** on the bus (**evt[2]=0**). With **evt[2]=1** the LIN endpoint checks each received message against the **byte_msg_payload** and if a match under the conditions given by **evt[1:0]** is found a reply is sent if **op=0**.

The LIN EP issues a trigger when a transmission has been finalized and the configured trailing time has expired. A LIN EP configured with a repeated trigger request on its own trigger can be used to cyclically send messages on the Lin bus.

13.7.10.2 LIN EP functional configuration

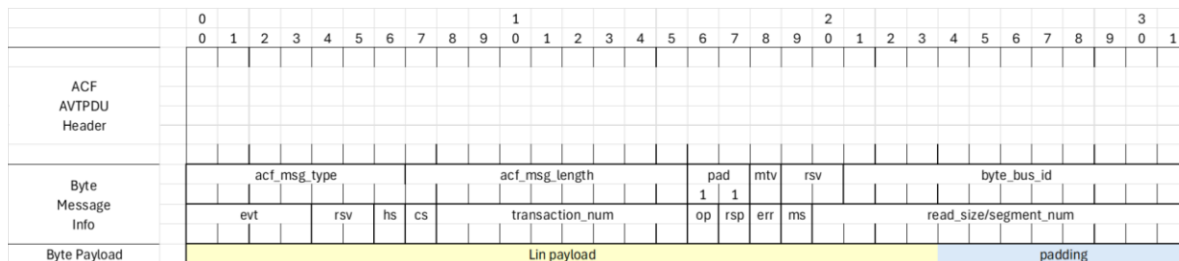
Functional configuration of the Lin endpoint.

Relative Address	Parameter	Type	R/W	Description
LIN				
0x0000	EP_LEN	8 bit	R	length of this description [0x04]
0x0001	LIN_BASE_CLK	16 bit	R	LIN system clock
0x0003	LIN_CLK_DIVIDER	8 bit	R/W	generate Lin bit time

13.7.10.3 LIN EP request handling

The Byte Msg Payload is the payload to be used on the Lin bus.

The example below shows 3 bytes payload.



13.7.11 Endpoint type: CAN controller

13.7.11.1 CAN EP basic concept

The CAN controller endpoint is receiving requests to either send a CAN (Classical CAN, CAN FD, CAN FD light or CAN XL - depending on endpoint capabilities) frame or wait for a specific payload to be received via CAN. This might be in combination with trigger conditions or as conditional execution in a compound request.

The CAN controller creates notifications upon reception of CAN frames.

13.7.11.2 CAN EP functional configuration

The functional configuration of the CAN controller comprises:

15. Bit rate settings for arbitration phase valid for CAN XL, CAN FD / CAN FD light and Classical CAN
16. Bit rate settings for CAN FD fast phase
17. Bit rate settings for CAN XL data phase
18. acceptance filter settings for CAN XL
19. receive ID filter settings (valid for all CAN variants)
20. usage of new PL (YES|NO) for CAN XL

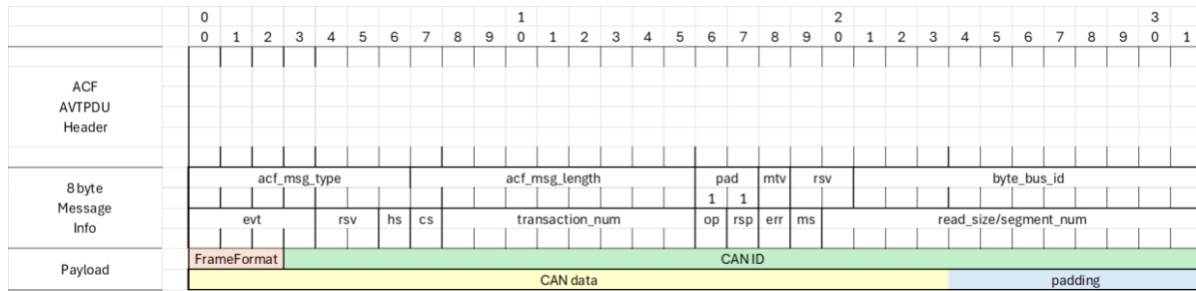
and may be extended depending on RC PHY implementation.

The functional configuration of the CAN endpoint only configures the endpoint clock, which is needed to define execution delays in triggered and compound operations.

relative Address	Parameter	Type	R/W	Description
CAN				
0x0000	EP_LEN	8 bit	R	length of this description [0x04]
0x0001	CAN_BASE_CLK	16 bit	R	CAN system clock
0x0003	CAN_CLK_DIVIDER	8 bit	R/W	generate CAN_CLK time reference
0x0004	CAN bit time register 1	32 bit	R/W	Defines bit times for Classical CAN
0x0008	CAN bit time register 2	32 bit	R/W	Defines bit times for CAN FD
0x000C	CAN bit time register 3	32 bit	R/W	Defines bit times for CAN XL
0x0010	TDCC register	32 bit	R/W	delay compensation control
0x0014	CAN EP status	32 bit	R/W	Status of CAN endpoint
0x0018	FIFO status	32 bit	R/W	Status of CAN FIFOs
0x001c	acceptance filter 1	32 bit	R/W	acceptance filter for CAN XL
0x0020	acceptance filter 2	32 bit	R/W	acceptance filter for CAN XL
0x0024	acceptance filter 3	32 bit	R/W	acceptance filter for CAN XL
0x0028	acceptance filter 4	32 bit	R/W	acceptance filter for CAN XL
0x002c	receive filter 1	32 bit	R/W	ID filter for frame reception
0x0030	receive filter 2	32 bit	R/W	ID filter for frame reception
0x0034	receive filter 3	32 bit	R/W	ID filter for frame reception
0x0038	receive filter 4	32 bit	R/W	ID filter for frame reception
...				

13.7.11.3 CAN EP request handling

The request and response format looks as follows:



The on-wire example shows "CAN data" with only 3 bytes. For CAN XL this can be up to 2054 bytes (2048 + 6, see below), which requires segmentation, which is supported by the fields "**ms**" and "**segment_num**".

The FrameFormat defines, whether the CAN frame is in CBFF, CEFF, FBFF, FEFF or XL (classical physical layer) or XL (XL physical layer).

CAN frame format	FrameFormat
CBFF	0
CEFF	1
FBFF	2
FEFF	3
XL (classic physical layer)	4
XL (new physical layer)	5
Reserved	6
reserved	7

CAN frame formats

Sending remote frames is not supported. In case the CAN ID is 11bits, then it shall be right aligned in the CAN ID field.

The "CAN data" field includes the additional data (RRS, SDT, VCID, AF (in total 6 bytes)- see ISO11898-1) needed for CAN XL frames, when the frame format is one of the two XL variants.

13.7.12 Endpoint type: ISELED

13.7.12.1 ISELED EP basic concept

The ISELED endpoint allows to remotely control an "ISELED Light and Sensor Network". This endpoint makes use of two physical pins and communicates bidirectional with the attached ISELED devices. Data sent from the RC Client is sent transparently to the ISELED network. Upon read requests the responses are collected and aggregated into a ACF type up to the requested **read_size**.

The ISELED endpoint creates a single trigger event, when transmission of a data packet has been completed on the ISELED network.

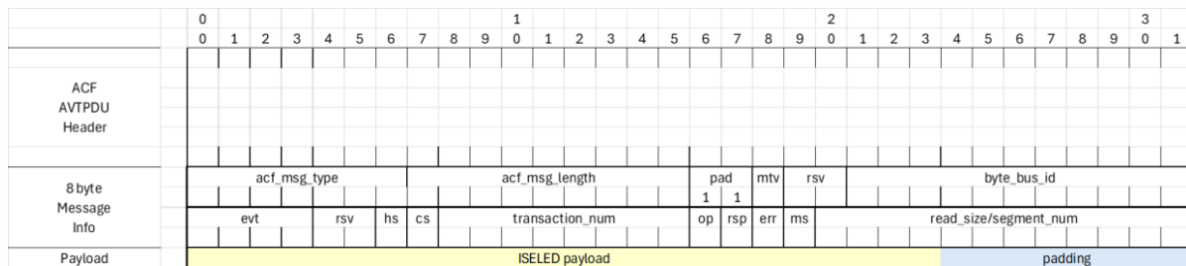
13.7.12.2 ISELED EP functional configuration

The functional configuration of the ISELED endpoint only configures the endpoint clock, which is needed to define execution delays in triggered and compound operations.

Relative Address	Parameter	Type	R/W	Description
ISELED				
0x0000	EP_LEN	8 bit	R	length of this description [0x04]
0x0001	ISELED_BASE_CLK	16 bit	R	ISELED system clock
0x0003	ISELED_CLK_DIVIDER	8 bit	R/W	generate ISELED_CLK time reference

13.7.12.3 ISELED EP request handling

The ISELED request, response and notification contains plain data in the **byte_msg_payload** that is to be presented or has been received on the ISELED bus.



On wire example showing 3 bytes of data plus one byte padding, just to illustrate the principle. Realistic data will have more bytes. In case of safe-operation mode a CRC will be added, that is not present on the ISELED network.